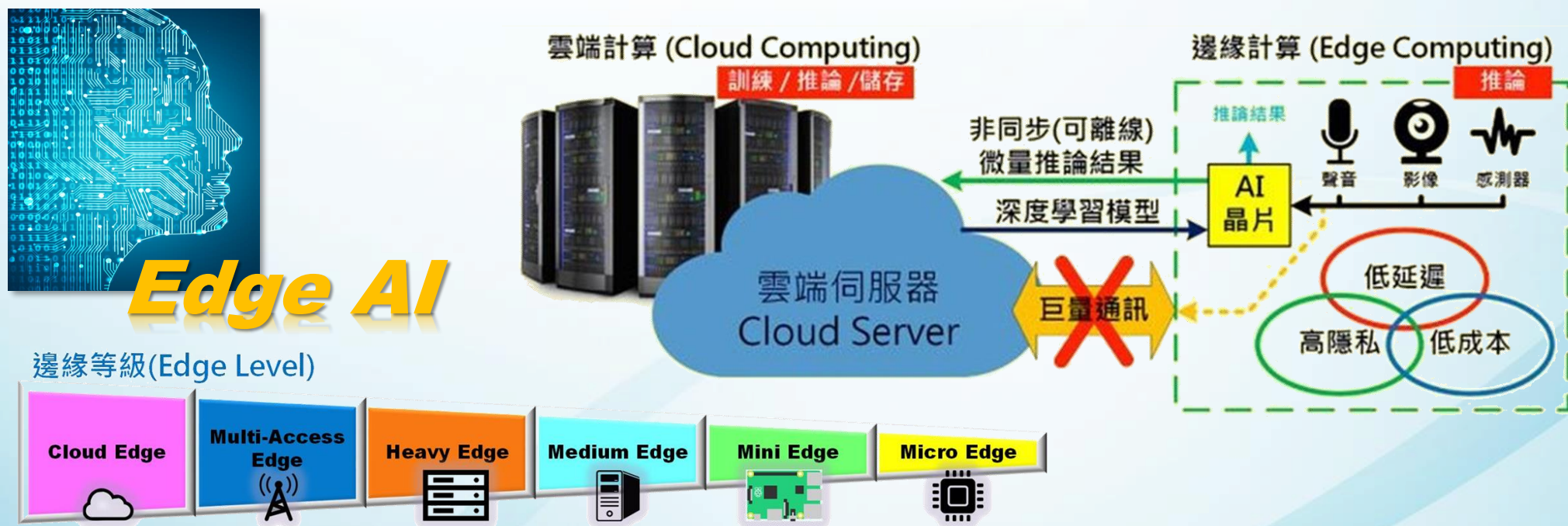


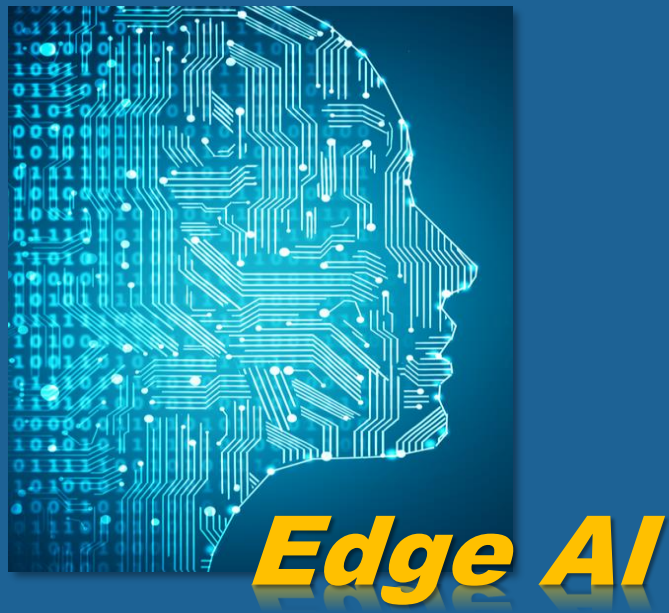
# 邊緣人工智慧實務 ( EE5354701 )



## 6 Edge AI 實例實作 — 影像辨識

電機工程系 人工智慧應用產碩專班 許哲豪 兼任助理教授

# 簡報大綱



## ➤ 6.1. 影像辨識簡介

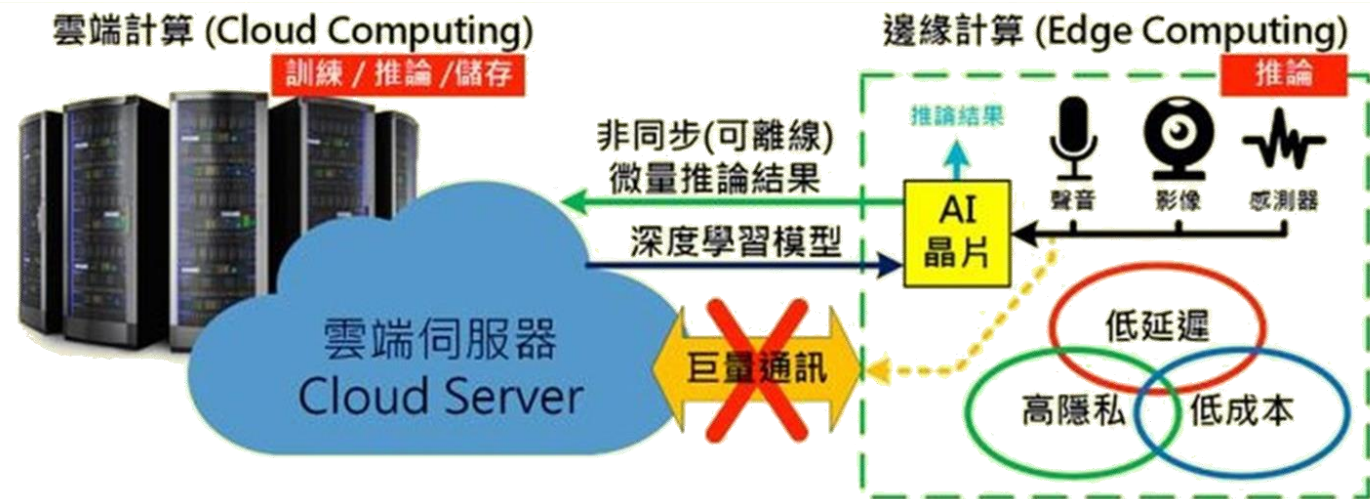
## ➤ 6.2. 影像分類

## ➤ 6.3. 物件偵測

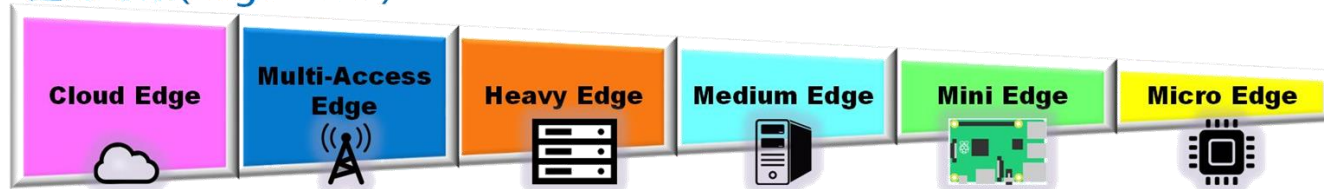
[https://hackmd.io/@OmniXRI-Jack/NTUST\\_EdgeAI\\_2026](https://hackmd.io/@OmniXRI-Jack/NTUST_EdgeAI_2026)



# Edge AI



邊緣等級(Edge Level)



## 6.1. 影像辨識簡介

# 傳統電腦 / 機器視覺 — 系統架構

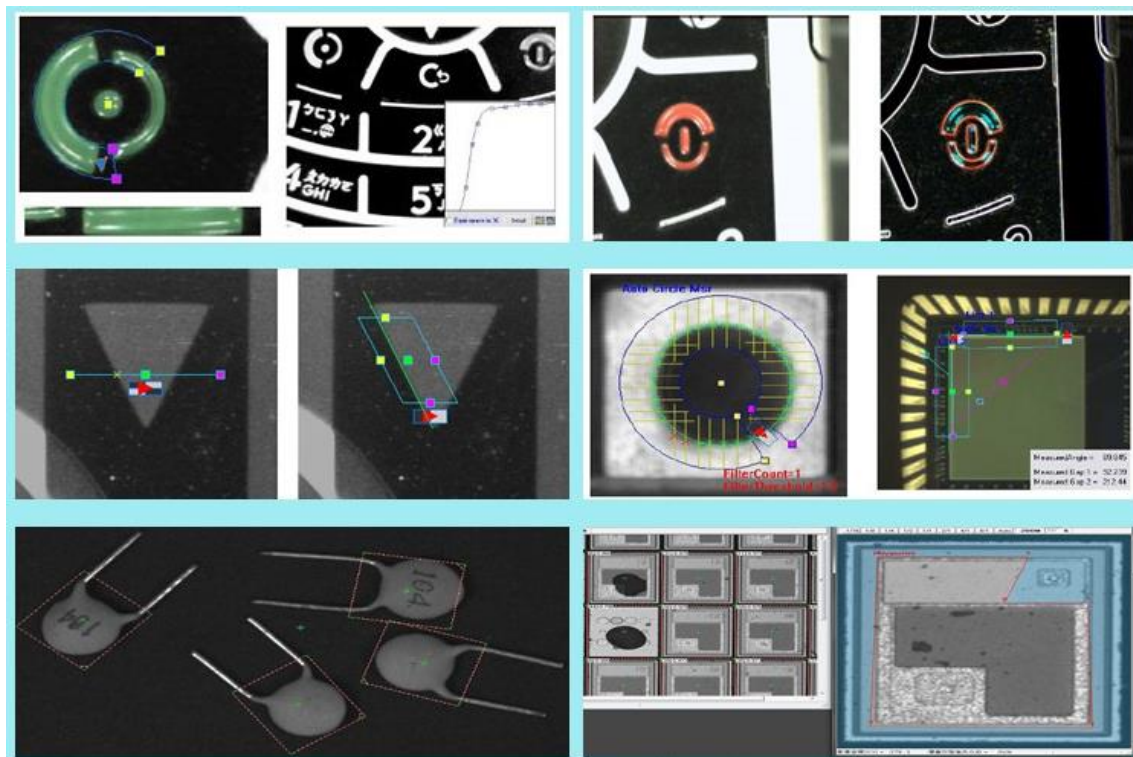


資料來源：<https://omnixri.blogspot.com/2021/07/edge-ai7-xai.html>

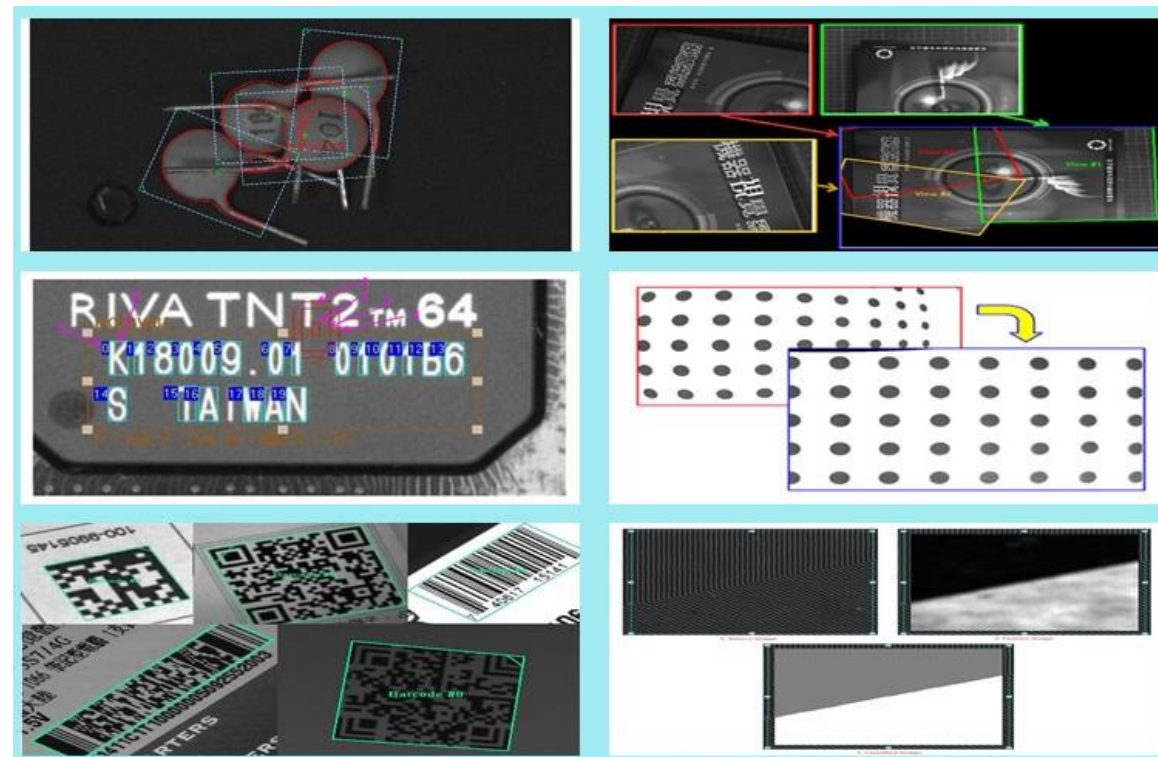
# 傳統電腦 / 機器視覺 — 自動光學檢測(AOI)

影像強化、特徵提取、瑕疵檢測、  
影像量測、區塊分析、圖樣比對

幾何比對、字元辨識、畸變校正、  
影像拼接、幾何分析、紋理分析



資料來源：<https://omnixri.blogspot.com/2021/07/edge-ai7-xai.html>



影像來源：光道視覺科技 <https://aisys.com.tw/web/about/>

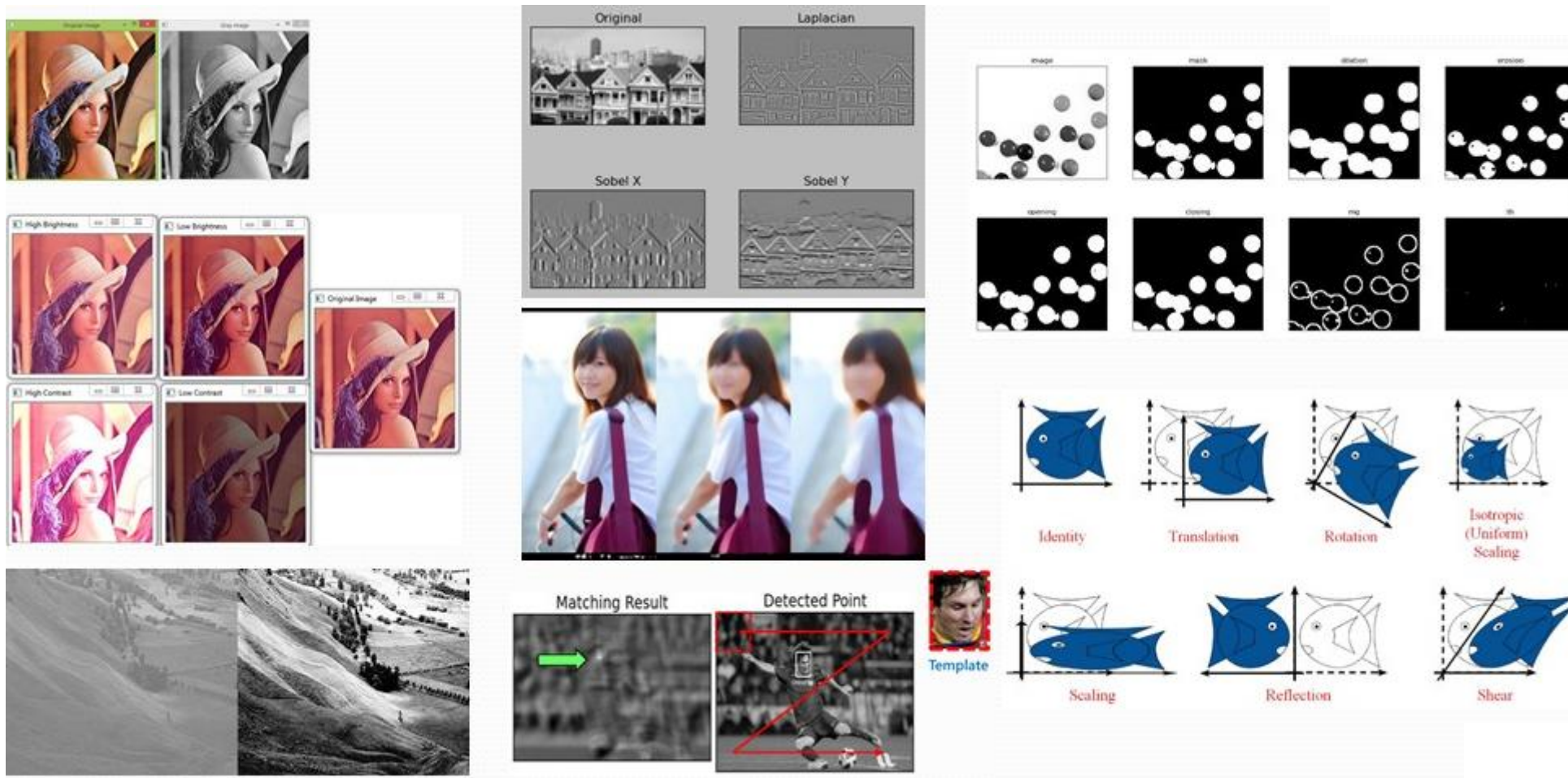
# 傳統電腦 / 機器視覺 — 開發工具 OpenCV (1/3)



**OpenCV** 是一個跨平台的電腦視覺庫，由英特爾公司發起並參與開發，以BSD授權條款授權發行，可以在商業和研究領域中免費使用。

OpenCV可用於開發即時的圖像處理、電腦視覺以及模式辨識程式。2011年起，OpenCV開始支援GPU即時加速運算(不包含GPU DNN加速)。

# 傳統電腦 / 機器視覺 — 開發工具 OpenCV (2/3)



- \* 影像調整  
( 亮度、對比等 )
- \* 影像濾波  
( 模糊、強化 )
- \* 邊緣提取
- \* 型態學
- \* 幾何畸變
- \* 模板比對搜尋  
( 物件偵測 )
- \* 基本繪圖

.....

# 傳統電腦 / 機器視覺 — 開發工具 OpenCV (3/3)



SLIC

SLICO



- \* 攝影機校正
- \* 立體視覺
- \* 深度影像
- \* 影像拼接
- \* 超像素分割
- \* 特徵點匹配
- \* 人臉定位
- \* 色彩物件追蹤

.....

# 範例6-1：Colab 中 OpenCV 顯示函式

如何在Colab上顯示雲端硬碟(Google Drive)上的影像和視頻

1. 將欲存取的檔案自行上傳到雲端硬碟(Google Drive)
2. 掛載自己的Google Drive
3. 選擇習慣之影像顯示方式
  - 方法1.1 透過matplotlib show()顯示
  - 方法1.2 透過IPython.display及PIL顯示
  - 方法1.3 以Colab自帶cv2\_imshow()函式顯示
4. 選擇習慣之視頻顯示方式
  - 方法2.1 透過IPython.display HTML()及base64 b64encode()函式庫顯示視頻
  - 方法2.2 利用ffmpeg處理OpenCV VideoWriter()產出視頻播放問題

[https://colab.research.google.com/github/OmniXRI/Colab\\_OpenCV\\_Display/blob/main/Colab\\_OpenCV\\_Display.ipynb#scrollTo=ziiOaC42vuRY](https://colab.research.google.com/github/OmniXRI/Colab_OpenCV_Display/blob/main/Colab_OpenCV_Display.ipynb#scrollTo=ziiOaC42vuRY)

# 常見 AI 影像辨識任務

物件偵測



- 多物件
- 物件框(座標、尺寸)
- 物件類別
- 置信度

影像分割



- 像素級分類
- 精準邊界
- 物件類別

影像分類



- 單一輸出
- 影像類別
- 置信度

姿態估測



- 關鍵點座標
- 自動隱藏遮蔽點

旋轉物件偵測



- 多物件
- 物件框(含旋轉角度)
- 物件類別
- 置信度

物件追蹤

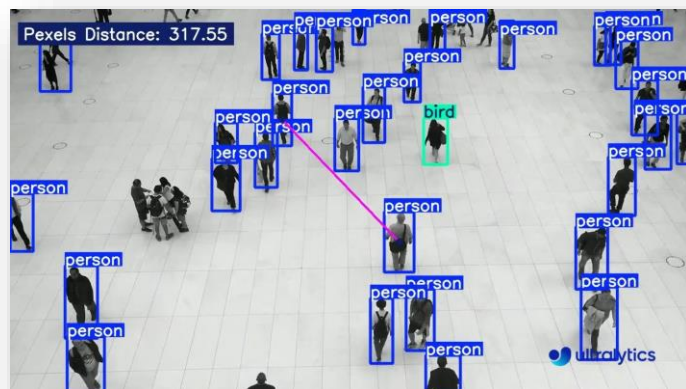


- 連續影像
- 物件編號
- 移動軌跡

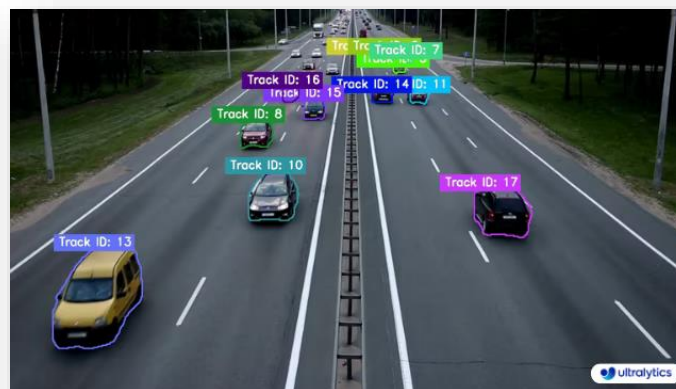
資料來源：<https://github.com/ultralytics/ultralytics>

# 常見 AI 影像辨識應用

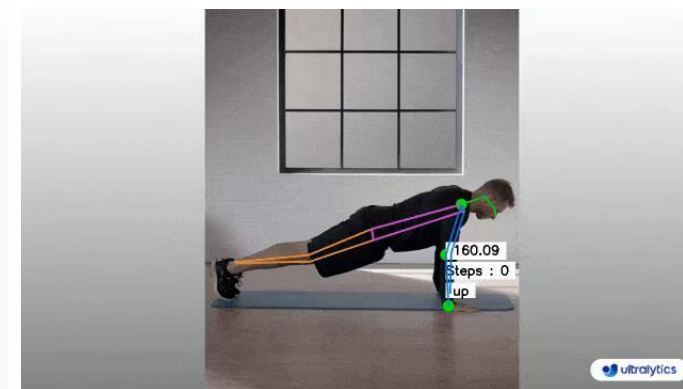
## 距離計算、移動速度



## 實例分割+物件追蹤



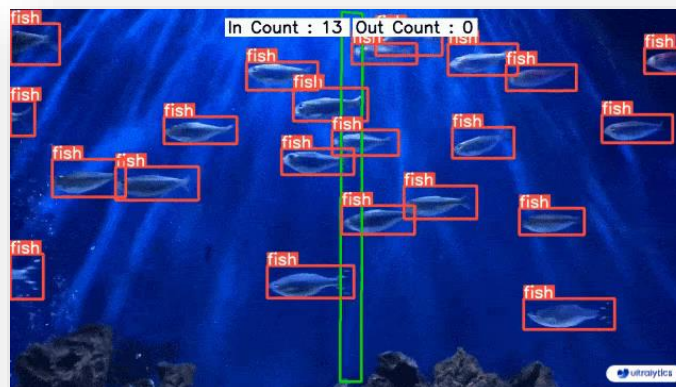
## 運動監測



## 熱力圖 ( 時序累積 )



## 物件計數、尺寸分級



## 停車管理



資料來源：<https://docs.ultralytics.com/zh/solutions/>

# 常見 AI 影像辨識開發工具

## 資料建置

- Roboflow
- Ultralytics
- Edge Impulse

## 模型訓練

- PyTorch
- TensorFlow
- Ultralytics (Yolo)
- Roboflow
- Edge Impulse
- Google Colab
- ONNX(僅輸出 )

## 部署優化

- Intel  
OpenVINO
- Nvidia  
TensorRT
- Google  
TensorFlowLite
- Edge Impulse

# 傳統型機器視覺 vs. 新型態AI影像辨識

| 項目        | 傳統機器視覺        | 人工智慧電腦視覺     |
|-----------|---------------|--------------|
| 取像環境要求    | 須精準控制光源、位置    | 不特別要求        |
| 分析方式      | 條件式、啟發式（專家規畫） | 深度學式模型（資料驅動） |
| 資料集 / 計算量 | 少量 / 中等       | 巨量 / 極巨量     |
| 精密尺寸量測    | OK            | NG           |
| 檢測條件模糊    | 過殺率過高         | 較穩定檢出        |
| 開發/佈署成本   | 高 / 中高        | 中等 / 極高      |

資料來源：<https://omnixri.blogspot.com/2021/07/edge-ai7-xai.html>

# Ultralytics 雲端訓練/推論平台(1/2)

## 1. 輸入個人基本資料

## 2. 選擇資料區域

## 3. 完成設定

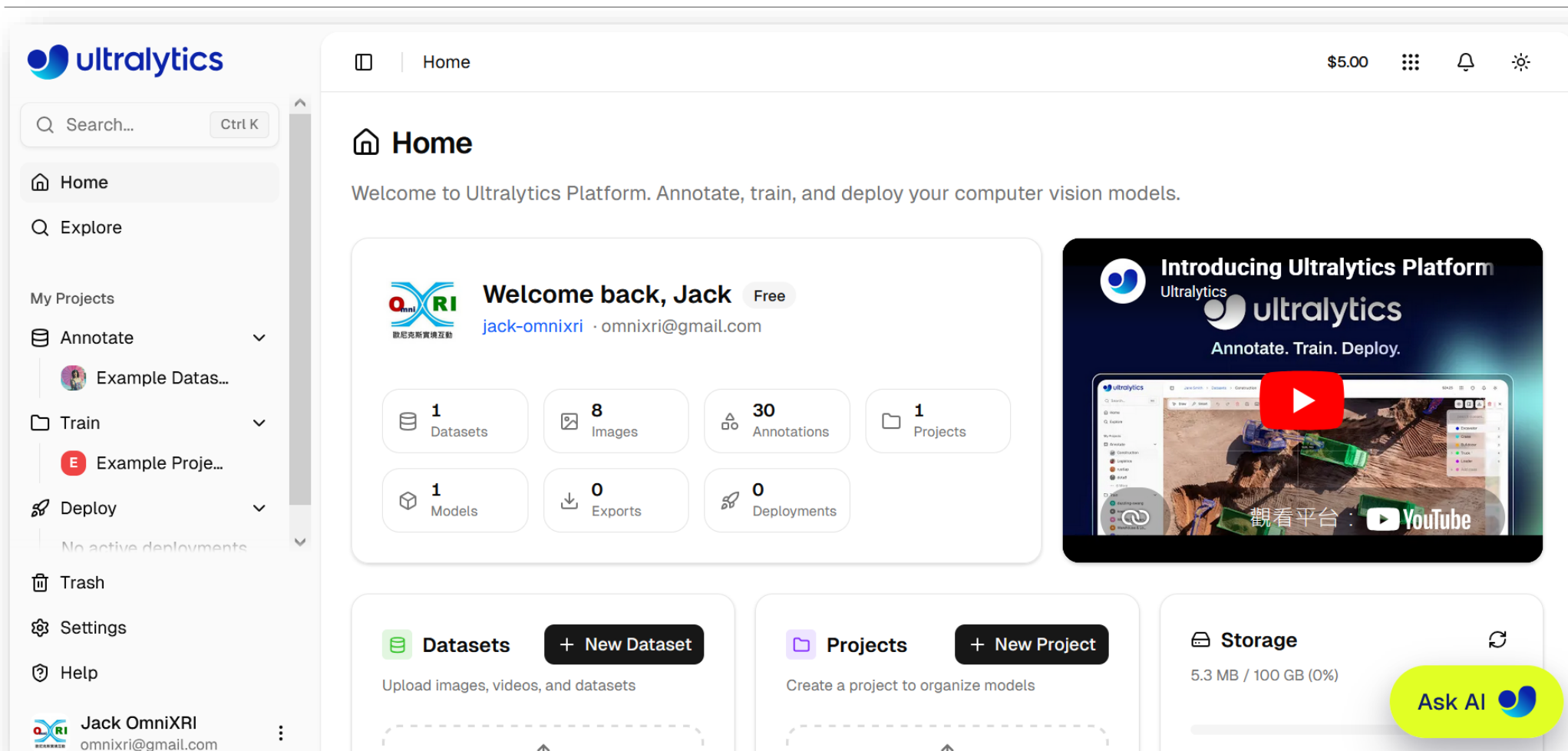
## GPU使用金額

| GPU          | 代次        | 显存     | 每小时费用  |
|--------------|-----------|--------|--------|
| RTX 2000 Ada | Ada       | 16 GB  | \$0.24 |
| RTX A4500    | Ampere    | 20 GB  | \$0.25 |
| RTX 4000 Ada | Ada       | 20 GB  | \$0.26 |
| RTX A5000    | Ampere    | 24 GB  | \$0.27 |
| L4           | Ada       | 24 GB  | \$0.39 |
| A40          | Ampere    | 48 GB  | \$0.40 |
| RTX 3090     | Ampere    | 24 GB  | \$0.46 |
| RTX A6000    | Ampere    | 48 GB  | \$0.49 |
| L40S         | Ada       | 48 GB  | \$0.86 |
| RTX 5090     | Blackwell | 32 GB  | \$0.89 |
| L40          | Ada       | 48 GB  | \$0.99 |
| A100 PCIe    | Ampere    | 80 GB  | \$1.39 |
| A100 SXM     | Ampere    | 80 GB  | \$1.49 |
| RTX PRO 6000 | Blackwell | 96 GB  | \$1.69 |
| H100 PCIe    | Hopper    | 80 GB  | \$2.39 |
| H100 SXM     | Hopper    | 80 GB  | \$2.69 |
| H100 NVL     | Hopper    | 94 GB  | \$3.07 |
| H200 NVL     | Hopper    | 143 GB | \$3.39 |
| H200 SXM     | Hopper    | 141 GB | \$3.59 |
| B200         | Blackwell | 180 GB | \$4.99 |

<https://platform.ultralytics.com>

可支援遠程 ( 本地端 ) GPU 訓練

# Ultralytics 雲端訓練/推論平台(2/2)



<https://platform.ultralytics.com>

# Roboflow 雲端訓練/推論平台

<https://app.roboflow.com>

## Plan & Billing

Change your plan or manage your billing settings

Your Plan

## Setting – Plan & Billing



### Public Plan

Best for personal, open source, and research projects, with public datasets and models for the community to use.

What's included:

15 credits per month

\$4 per additional credit

Public data

Community Support

Change Plan

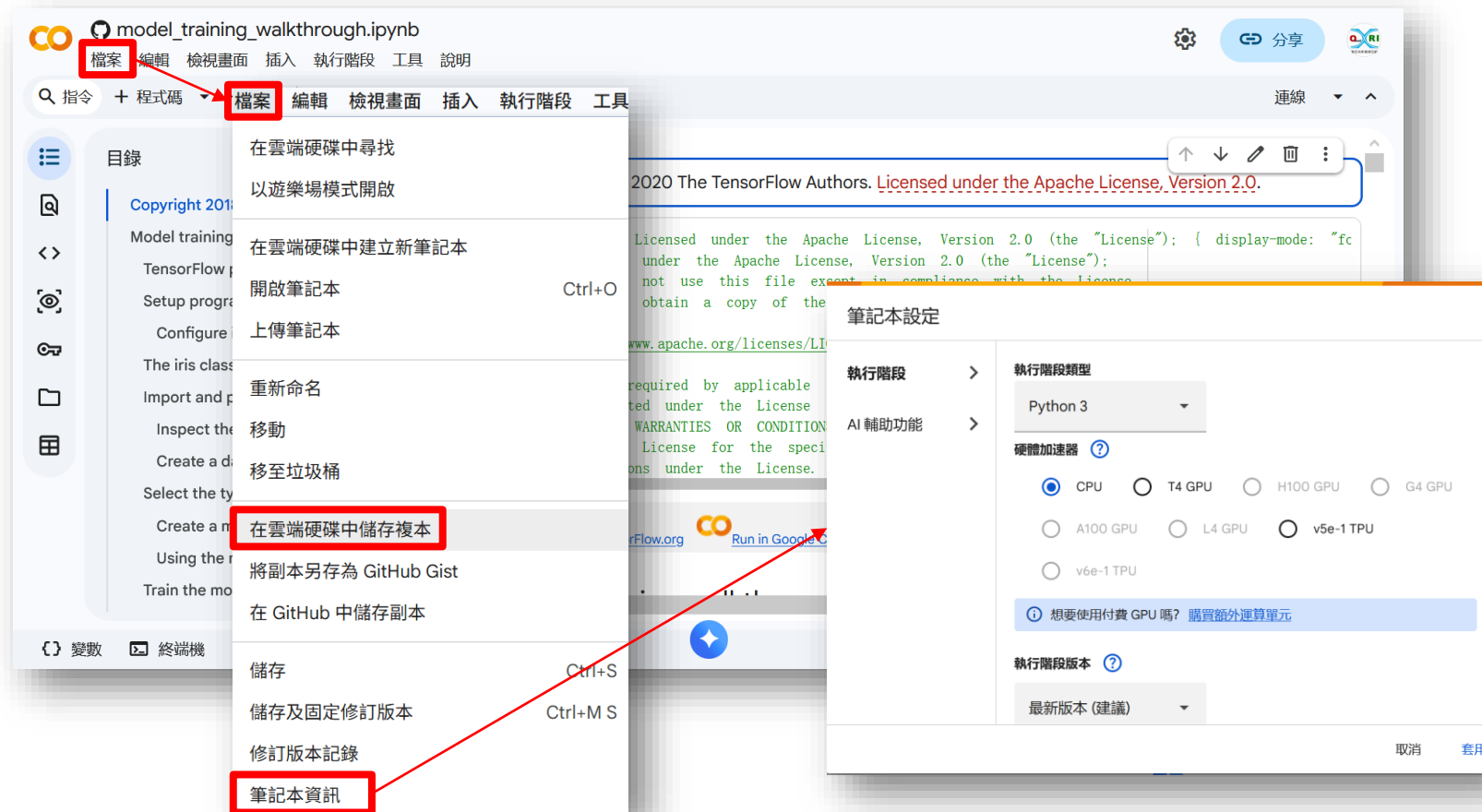
**免費訓練額度**

**每月15點 ( 1點為4美金 )**

**資料集要公開、適用小模型及資料集**

**可搭配Google Colab 免費訓練**

# Google Colab 雲端訓練/推論平台



- Jupyter Notebook環境
- 已預安裝好很多常用函式庫
- 免費 Google TPU / Nvidia GPU, 12GB RAM
- 100GB 虛擬機硬碟 (逾時或結束連線時會自動清空)
- 可連動 Google Drive
- 單次最長12小時 (理論值)
- 動態配置，當使用者多時，會自動斷線，須搭配 Google Drive 定期儲存訓練結果，以免全部消失。
- 使用量高時會被鎖帳號，停用1天到7天不等。
- 可訂閱付費版 Colab Pro 獲得更多計算時間。

<https://colab.research.google.com/>

# 如何使用 Colab 直接開啟 Github 程式

假設 Github 中有一個 [yolov26-object-detection-pytorch.ipynb](#)

點選右鍵查看原始鏈結為

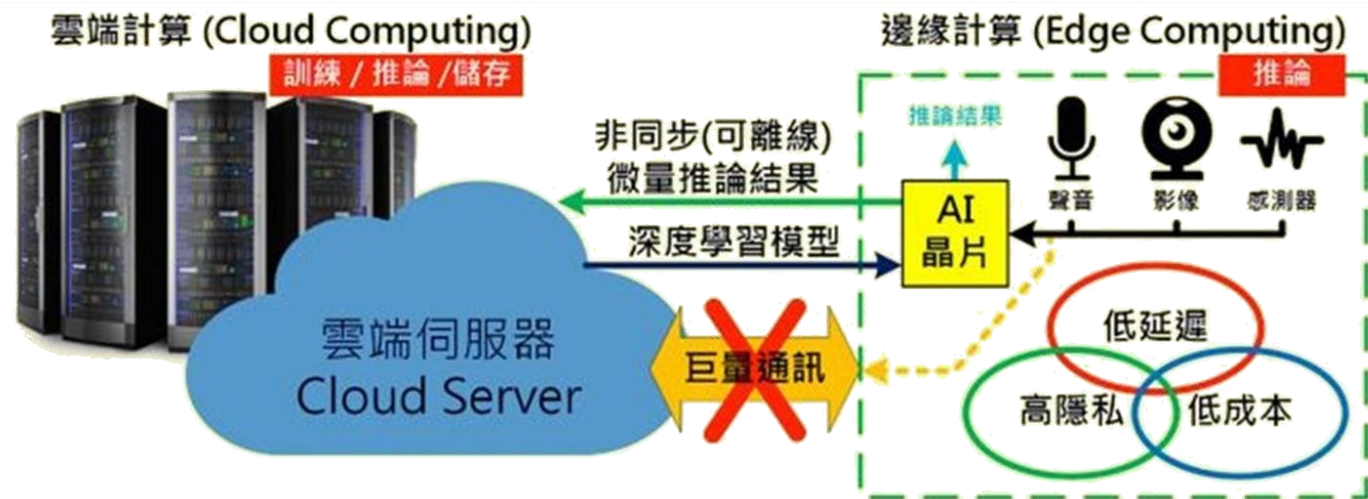
[https://github.com/OmniXRI/NTUST\\_EdgeAI\\_2026/blob/main/Ch6\\_Image\\_Recognition/yolov26-object-detection-pytorch.ipynb](https://github.com/OmniXRI/NTUST_EdgeAI_2026/blob/main/Ch6_Image_Recognition/yolov26-object-detection-pytorch.ipynb)

此時要將路徑修改如下才能直接點擊執行

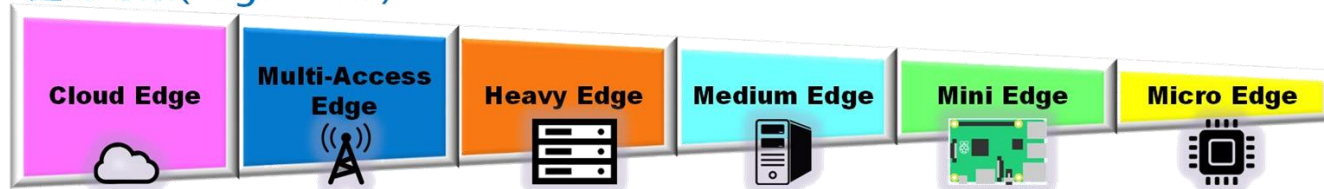
[https://colab.research.google.com/github/OmniXRI/NTUST\\_EdgeAI\\_2026/blob/main/Ch6\\_Image\\_Recognition/yolov26-object-detection-pytorch.ipynb](https://colab.research.google.com/github/OmniXRI/NTUST_EdgeAI_2026/blob/main/Ch6_Image_Recognition/yolov26-object-detection-pytorch.ipynb)



## Edge AI



### 邊緣等級(Edge Level)



## 6.2. 影像分類

# 常見電腦視覺任務

## 影像分類

### Classification



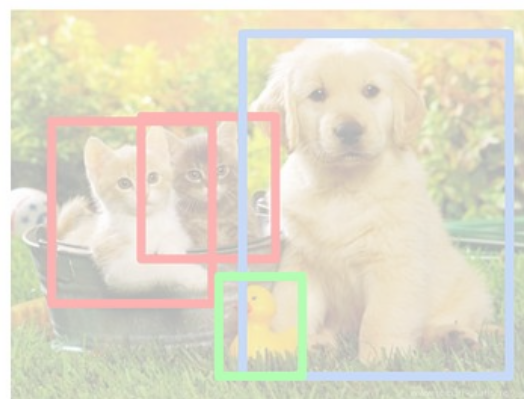
CAT

### Classification + Localization



CAT

### Object Detection



CAT, DOG, DUCK

### Instance Segmentation



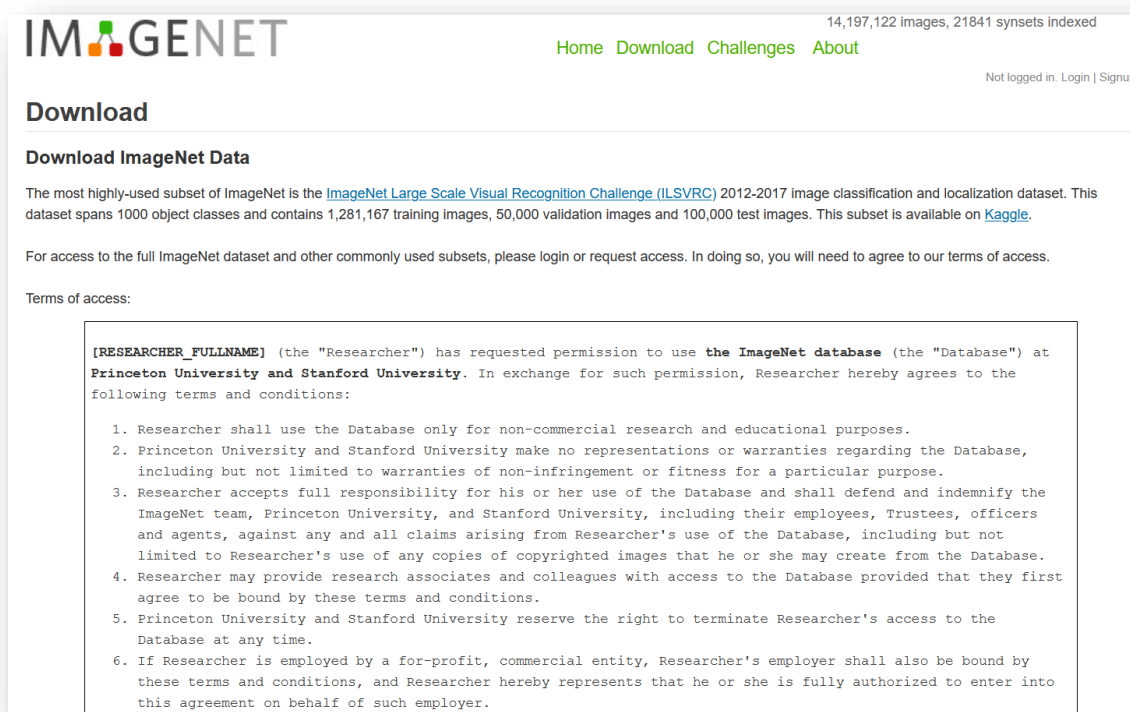
CAT, DOG, DUCK

Single object

Multiple objects

# 常見影像分類資料集 - ImageNet

**ImageNet** 原始資料集共有**1400**多萬張影像，共分為**2萬1千**多類，另有**1000**分類版。可應用於影像分類及物件偵測等電腦視覺任務。



<https://www.image-net.org/index.php>

# 常見影像分類模型

## 卷積型

- ResNet
- VGG
- Inception (GoogLeNet)

## 輕量化

- MobileNet (V1/V2/V3)
- EfficientNet (V1/V2)
- ConvNeXt (V1/V2)

## ViT 型

- ViT (Vision Transformer)
- Swin Transformer

### 效能評比基準

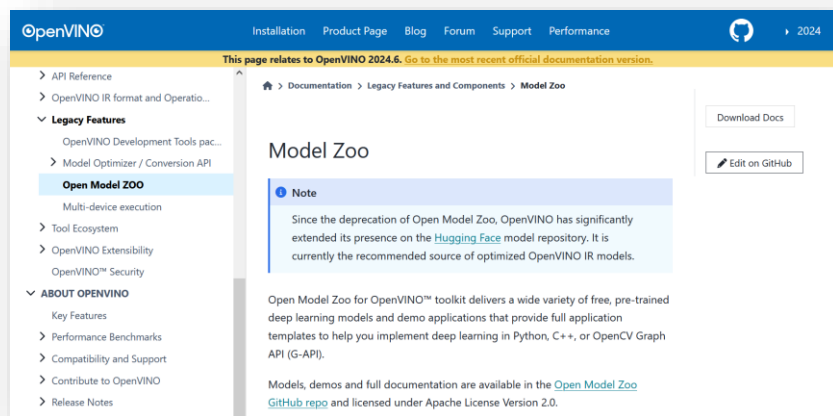
- TOP-1 Accuracy
- TOP-5 Accuracy

### 推論考量重點

- 推論精度
- 推論速度
- 模型大小

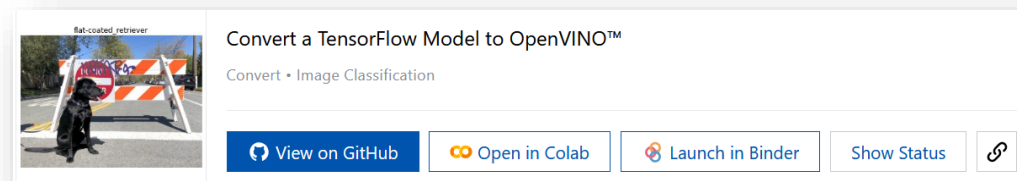
# Intel OpenVINO Model Zoo & Converter

**OpenVINO Model Zoo** 保留常用 CNN 各類模型，主要分為 **Intel Pre-Trained** (可直接調用 **IR** 即 **\*.bin + \*.xml**) 和 **Public** (需自行使用 Converter 將各種框架模型轉換成 IR 格式)。官方文件雖已歸入 Legacy Function，但 Github 仍保留所有項目)



各種框架產出已訓練好的模型，都可透過 OpenVINO 對應的 Converter 進行轉換，包括：

- TensorFlow
- PaddlePaddle
- PyTorch
- TensorFlowLite



[https://github.com/openvinotoolkit/open\\_model\\_zoo/](https://github.com/openvinotoolkit/open_model_zoo/)

# OpenVINO Model Zoo - Public - Classification

|                         |                      |                         |                                   |                       |                               |                              |
|-------------------------|----------------------|-------------------------|-----------------------------------|-----------------------|-------------------------------|------------------------------|
| alexnet                 | efficientnet-v2-s    | inception-resnet-v2-tf  | mobilenet-v2-pytorch              | regnetx-3.2gf         | resnet-34-pytorch             | shufflenet-v2-x1.0           |
| anti-spoof-mn3          | hbonet-1.0           | levit-128s              | mobilenet-v2-1.4-224              | octave-resnet-26-0.25 | resnet-50-pytorchresnet-50-tf | squeezenet1.0                |
| caffenet                | hbonet-0.25          | mixnet-l                | mobilenet-v3-small-1.0-224-tf     | open-closed-eye-0001  | rexnet-v1-x1.0                | squeezenet1.1                |
| convnext-tiny           | googlenet-v1         | mobilenet-v1-0.25-128   | mobilenet-v3-small-1.0-paddle     | repvgg-a0             | se-inception                  | swin-tiny-patch4-window7-224 |
| densenet-121            | googlenet-v1-tf      | mobilenet-v1-1.0-224    | mobilenet-v3-large-1.0-224-tf     | repvgg-b1             | se-resnet-50                  |                              |
| densenet-121-tf         | googlenet-v2         | mobilenet-v1-1.0-224-tf | mobilenet-v3-large-1.0-224-paddle | repvgg-b3             | se-resnext-50                 | t2t-vit-14                   |
| dla-34                  | googlenet-v2-tf      | mobilenet-v2            | mobilenet-v2-1.0-224              | resnest-50-pytorch    | shufflenet-v2-x0.5            | vgg16                        |
| efficientnet-b0         | googlenet-v3         | mobilenet-v2            | mobilenet-v2-1.0-224              | resnet-18-pytorch     |                               | vgg19                        |
| efficientnet-b0-pytorch | googlenet-v3-pytorch | mobilenet-v2            | mobilenet-v2-1.0-224              |                       |                               |                              |
| efficientnet-v2-b0      | googlenet-v4-tf      | mobilenet-v2            | mobilenet-v2-1.0-224              |                       |                               |                              |

[https://docs.openvino.ai/2023.3/omz\\_models\\_group\\_public.html#classification-models](https://docs.openvino.ai/2023.3/omz_models_group_public.html#classification-models)

# OpenVINO 影像分類 Colab 範例

## hello-world.ipynb



使用 Intel Pre-Trained **MobileNetV3** 模型, ImageNet 1000 分類預訓練。

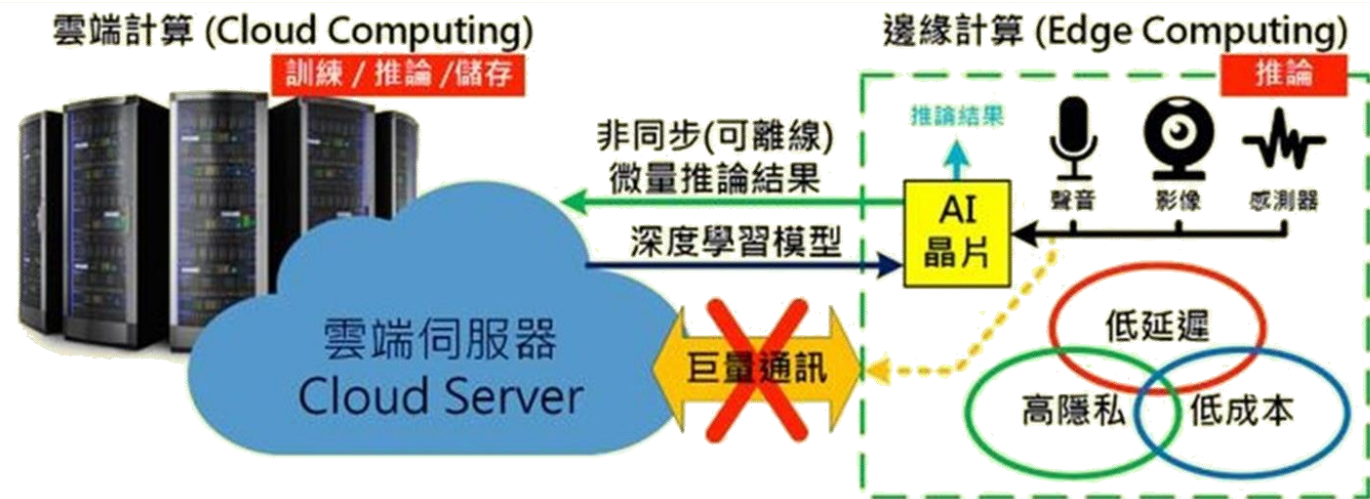
- Imports
- Download the Model and data samples
- Select inference device (CPU/GPU/AUTO)
- Load the Model
- Load an Image (Convert RGB to BRG, Resize)
- Do Inference

**'n02099267 flat-coated retriever'**

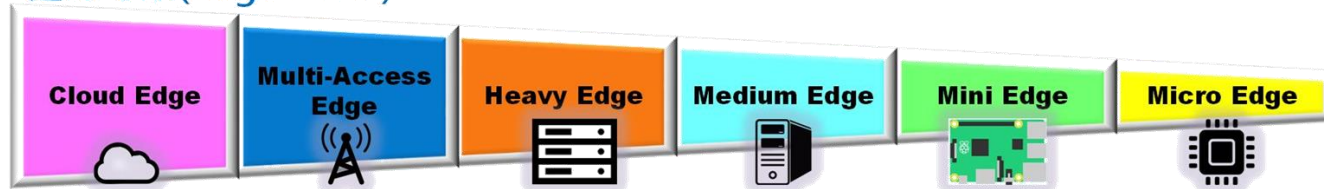
[https://colab.research.google.com/github/openvinotoolkit/openvino\\_notebooks/blob/latest/notebooks/hello-world/hello-world.ipynb](https://colab.research.google.com/github/openvinotoolkit/openvino_notebooks/blob/latest/notebooks/hello-world/hello-world.ipynb)



# Edge AI

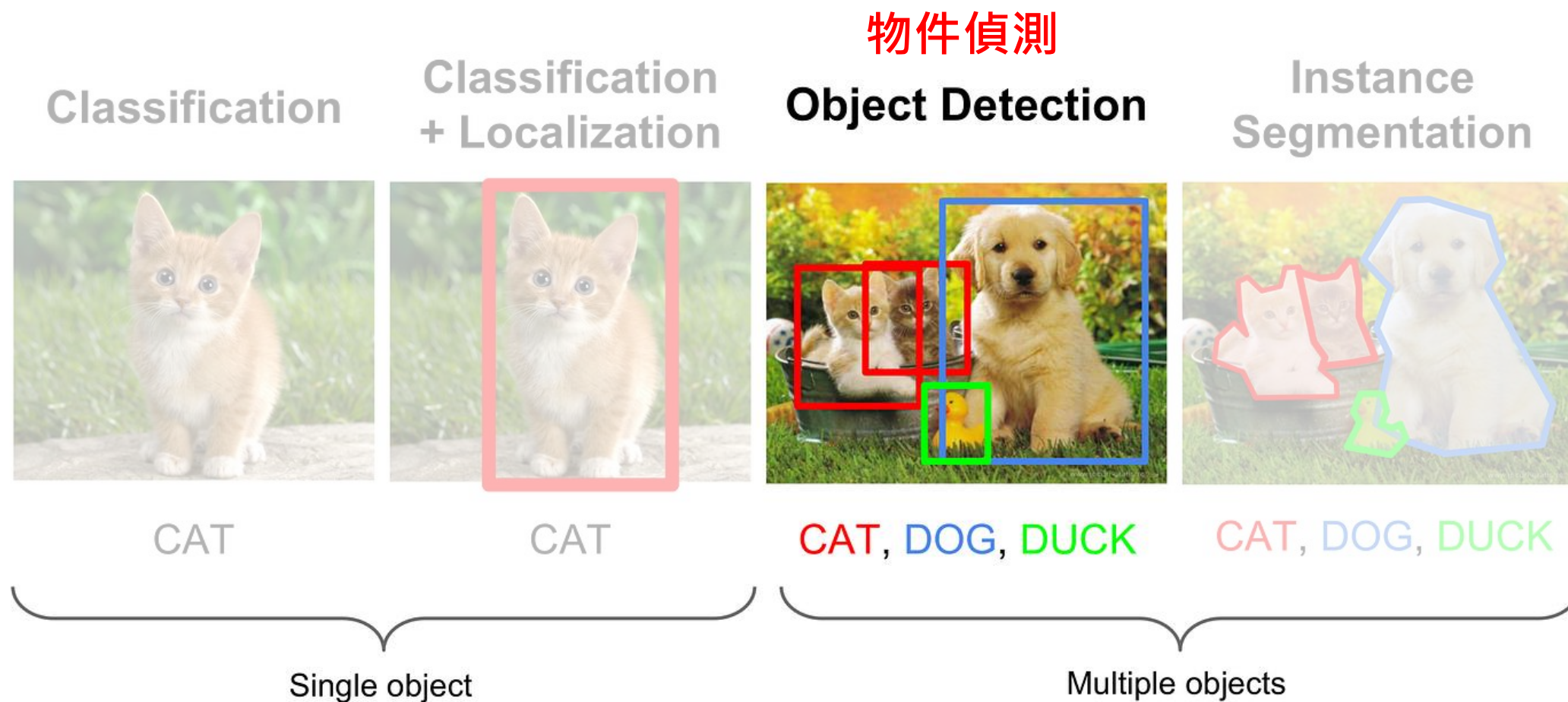


邊緣等級(Edge Level)

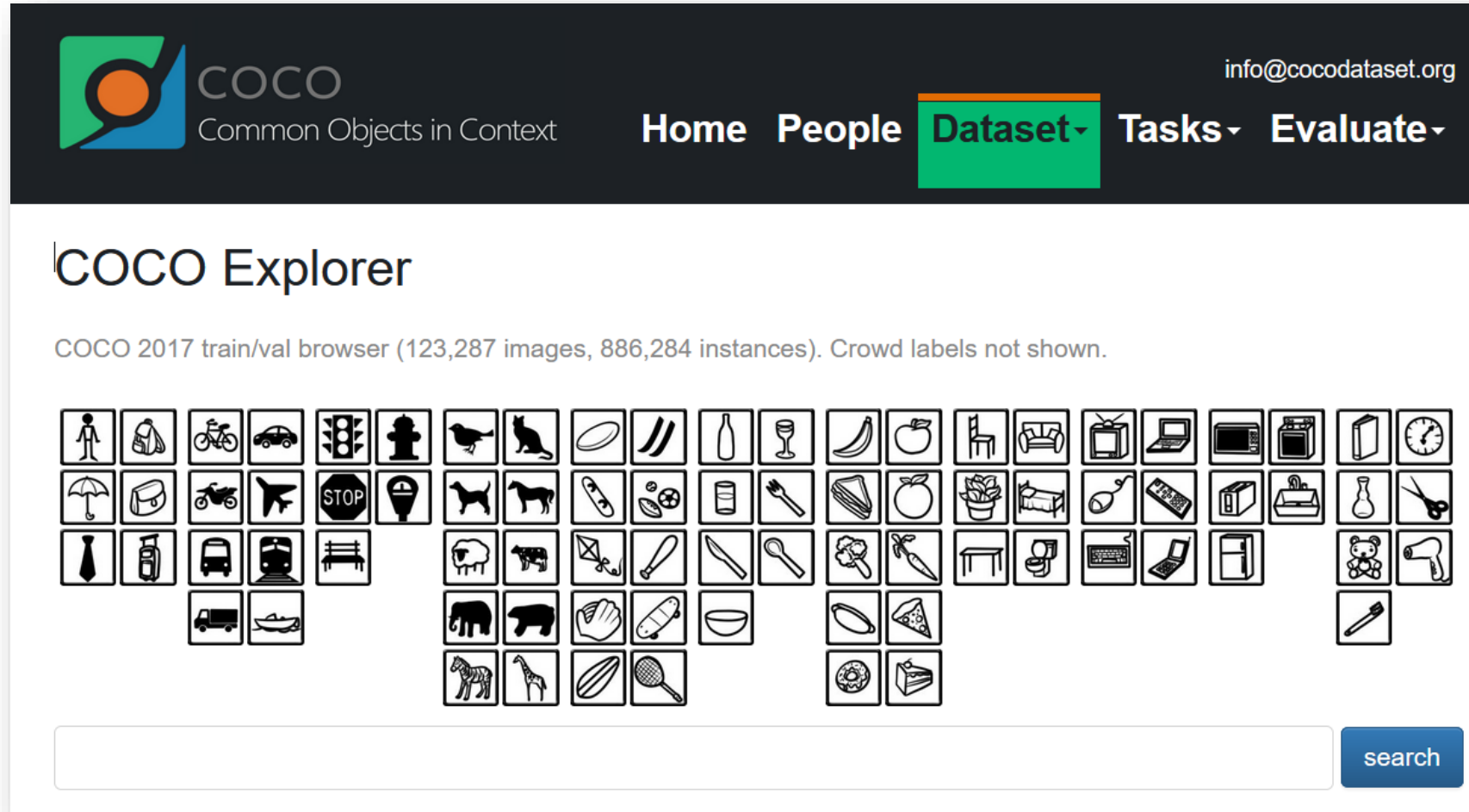


## 6.3. 物件偵測

# 常見電腦視覺任務



# 常見物件偵測資料集 – Microsoft COCO

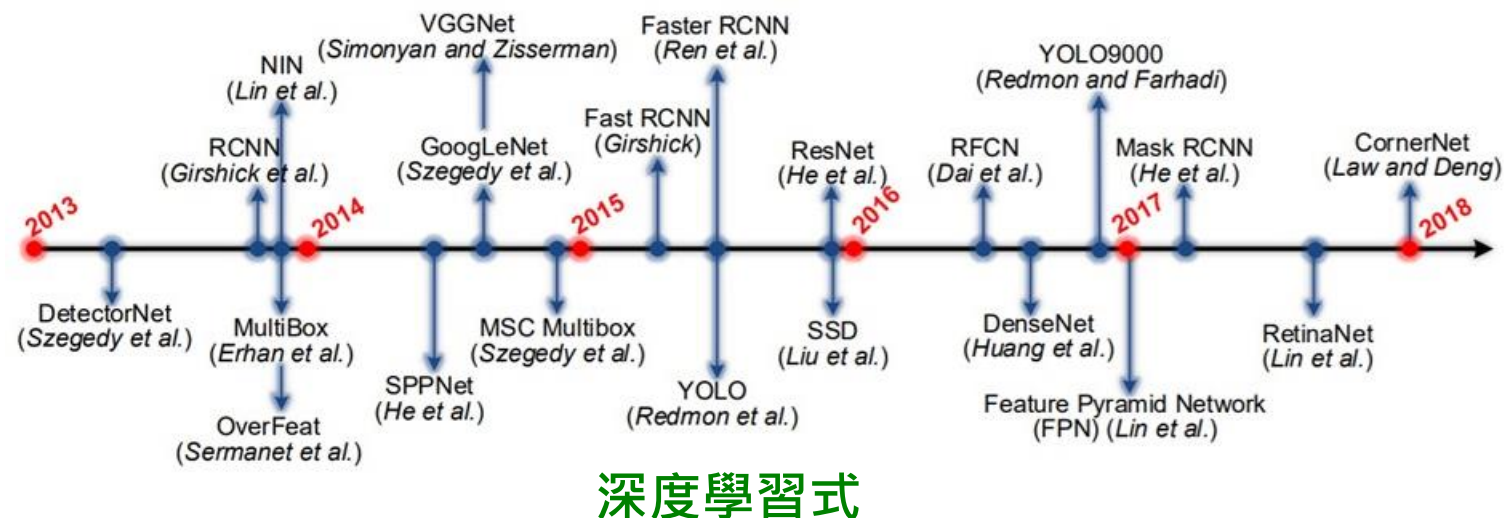
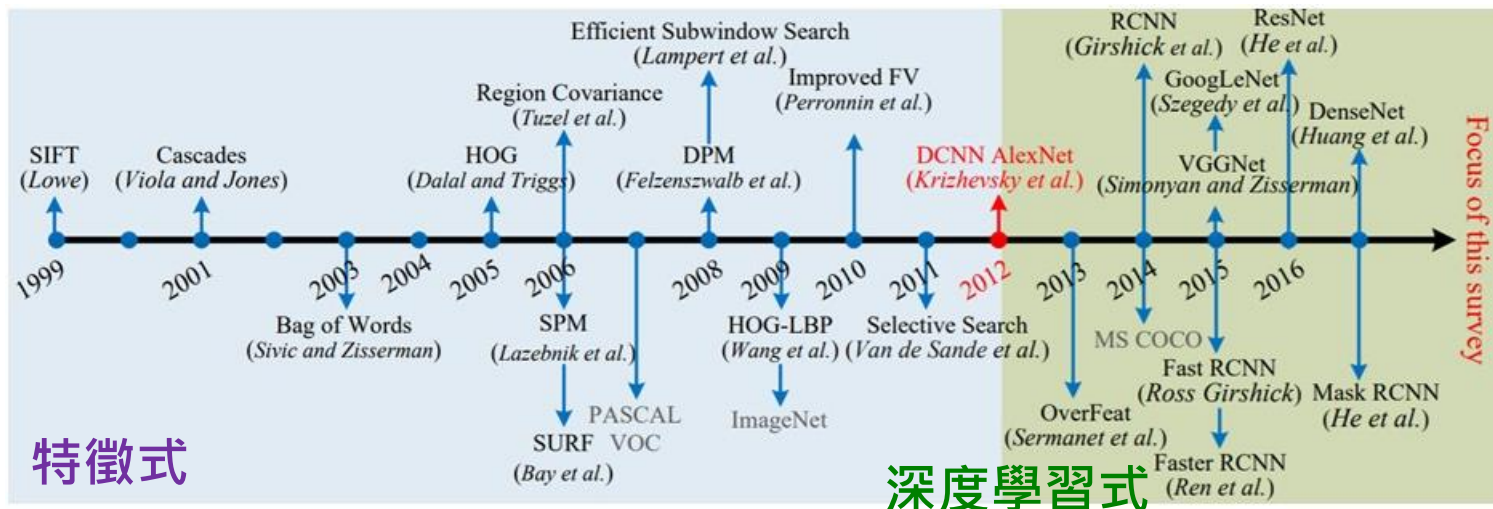


常見物件共80類，包括日常生活常見物品。

資料集包括物件偵測、實例分割、圖像描述及關鍵點偵測。

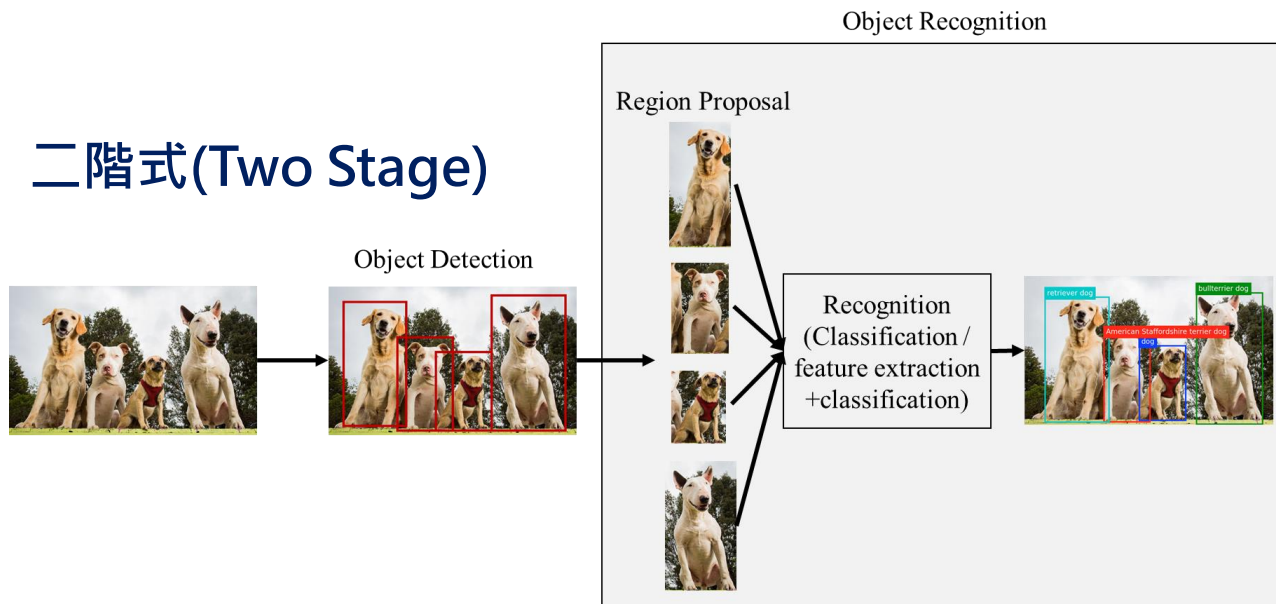
<https://cocodataset.org/#explore>

# 物件偵測技術演進



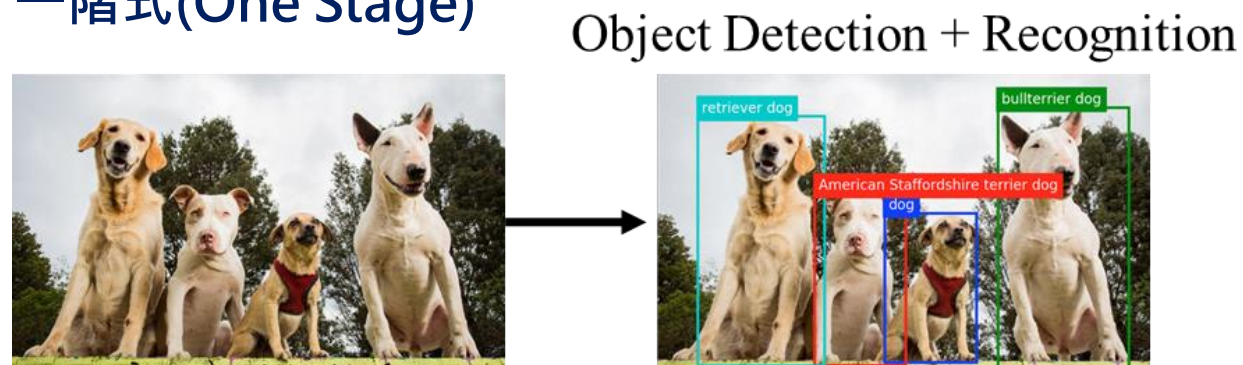
# 常見物件偵測模型

## 二階式(Two Stage)



先找出物件可能區域(Region Proposals)再進行物件辨識，如  
**R-CNN**  
**Fast R-CNN**  
**Faster R-CNN**  
 ...

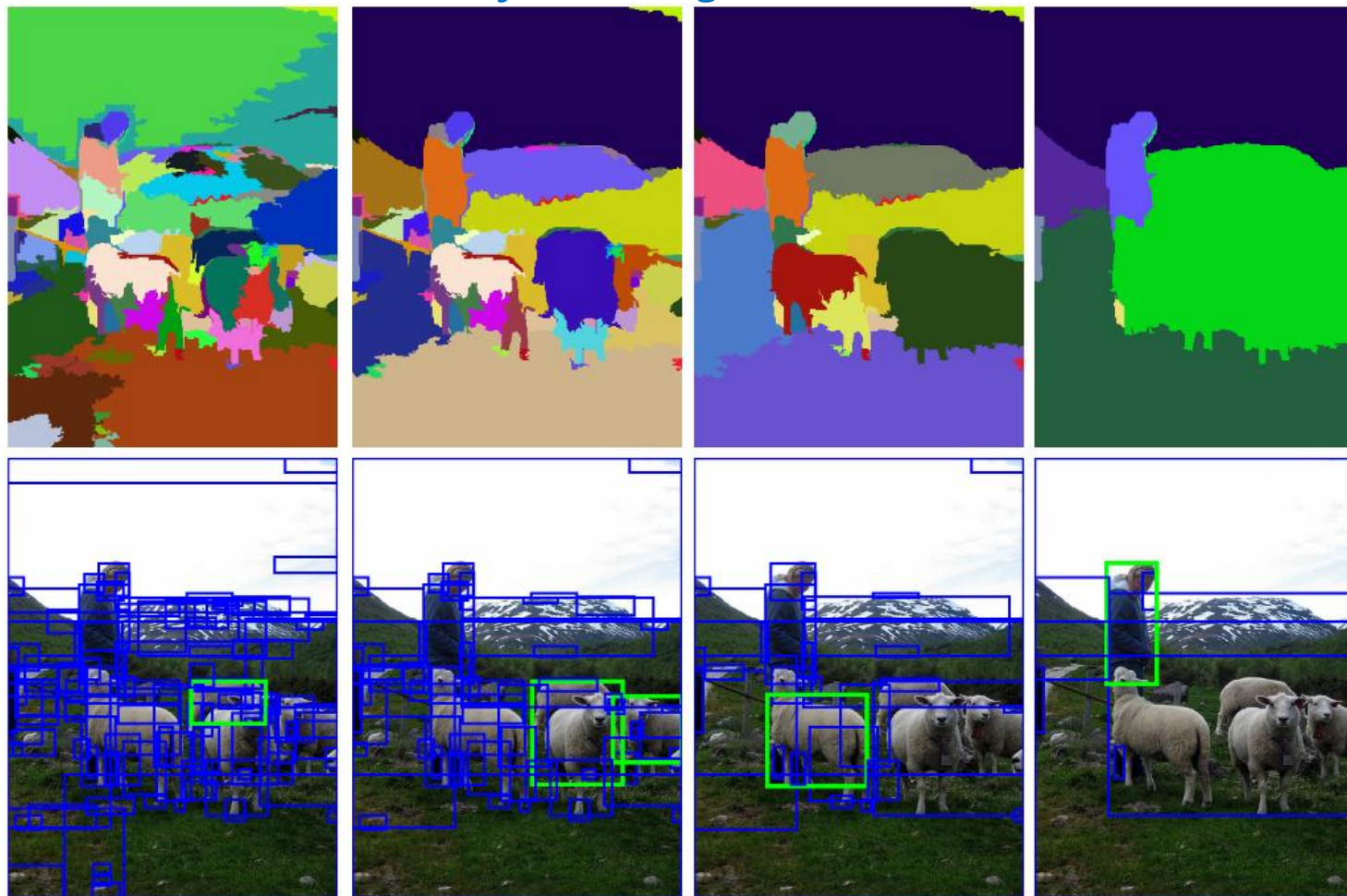
## 一階式(One Stage)



物件偵測及辨識一起完成，如  
**SSD**  
**YOLO**  
 ...

# 候選區域(Region Proposals)

## Selective Search for Object Recognition



二階式  
物件偵測

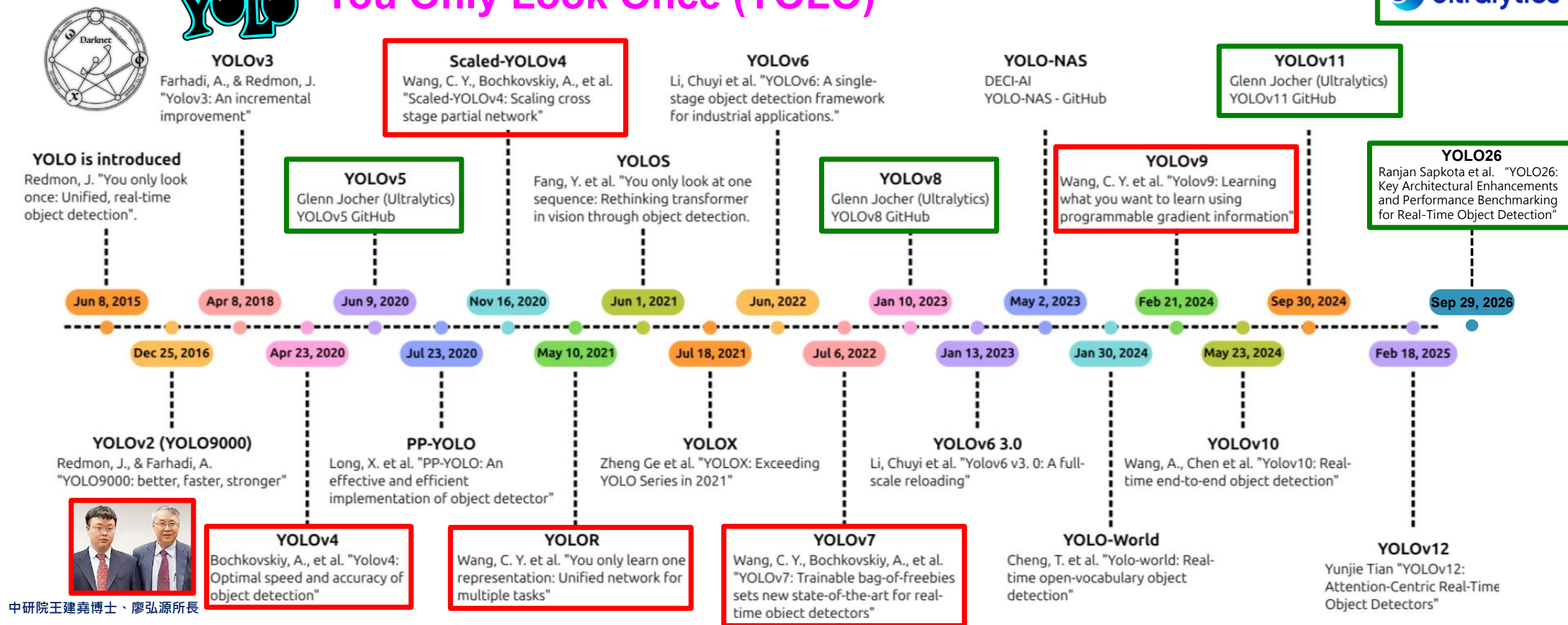
第一階：  
產生候選  
區域再合  
併。

資料來源：<http://www.huppelen.nl/publications/selectiveSearchDraft.pdf>

# 一階式物件偵測 YOLO 發展歷程



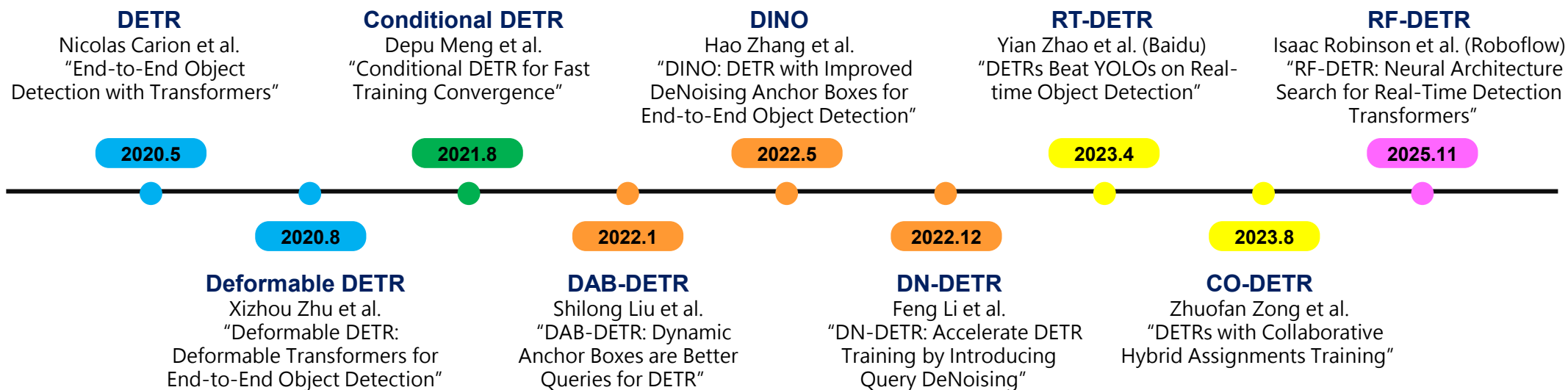
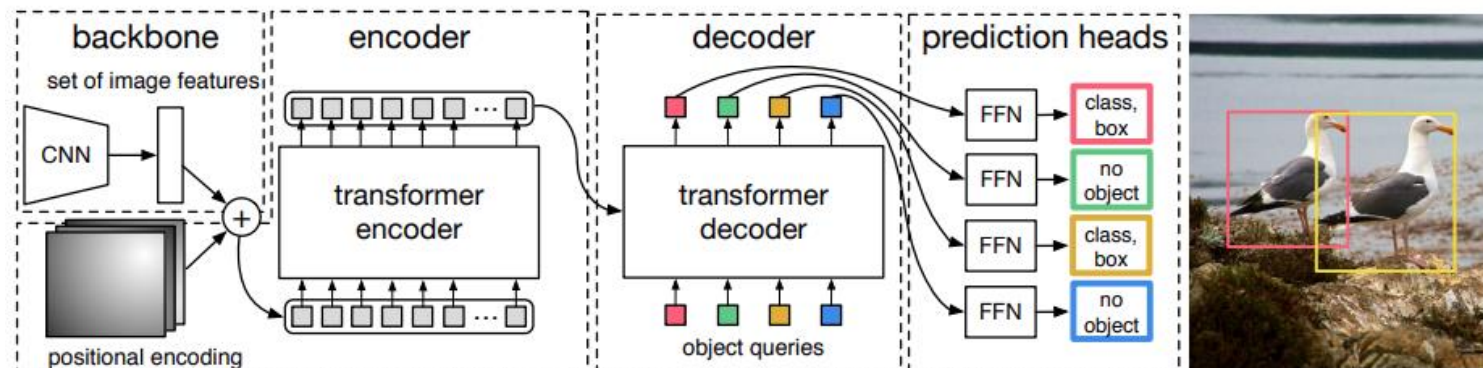
You Only Look Once (YOLO)



中研院王建義博士、廖弘源所長

資料來源：<https://www.themoonlight.io/tw/review/yolo-evolution-a-comprehensive-benchmark-and-architectural-review-of-yolov12-yolov11-and-their-previous-versions>

# 基於 Transformer 物件偵測技術



# 旋轉物件偵測

Ultralytics Yolo v8/v11/26 皆有提供旋轉物件偵測(OBB)功能，主要搭配 DOTAv1 數據集進行預訓練，可偵測常見十多種空拍建物。資料集表示方式為

```
x1, y1, x2, y2, x3, y3, x4, y4, category, difficult
x1, y1, x2, y2, x3, y3, x4, y4, category, difficult
...
```



<https://captain-whu.github.io/DOTA/index.html>

# 常見物件偵測資料集格式

## VOC 物件偵測資料格式

```
<size> 影像尺寸
  <width>353</width> 影像寬度
  <height>500</height> 影像高度
  <depth>3</depth> 色彩深度(通道數)
</size>
<segmented>0</segmented> 是否分割
<object> 物件(目標)訊息
  <name>dog</name> 物件名稱(20分類)
  <pose>Left</pose> 拍攝角度(前後左右及未定)
  <truncated>1</truncated> 是否被遮擋或截斷
  <difficult>0</difficult> 是否難以檢測
  <bndbox> 物件(目標)外框
    <xmin>48</xmin> 物件左上角X座標
    <ymin>240</ymin> 物件左上角Y座標
    <xmax>195</xmax> 物件右下角X座標
    <ymax>371</ymax> 物件右下角Y座標
  </bndbox>
</object>
```

邊界框位置  
以左上表示

## YOLO 物件偵測資料轉換

$$X = (xmin + (xmax - xmin) / 2) * 1.0 / width$$

$$Y = (ymin + (ymax - ymin) / 2) * 1.0 / height$$

$$W = (xmax - xmin) * 1.0 / width$$

$$H = (ymax - ymin) * 1.0 / height$$

邊界框位置  
以中心表示

## YOLO 物件標註檔案格式(\*.txt)

[分類編號ID] [物件中心X座標] [物件中心Y座標] [物件寬度佔比W] [物件高度佔比H]  
0 0.344192634561 0.611000000000 0.416430594901 0.262000000000

OmniXRI Oct. 2020 整理繪製

# Ultralytics YOLO

ultralytics

Ultralytics 简介: 标注、训练和部署YOLO

ultralytics

Ultralytics YOLO 文档

ultralytics/ultralytics

v8.4.37 ☆ 56k 10.8k

README

Code of conduct

Contributing

AGPL-3.0 license

Security

快速入门

用法

CLI

Python

回调函数

配置

简单实用工具

高级自定义

YOLO26

The ultimate end-to-end vision AI platform

Get started

中文 · 한국어 · 日本語 · Русский · Deutsch · Français · Español · Português · Türkçe · Tiếng Việt

CI passing downloads 177M DOI 10.5281/zenodo.7347926 Discord 721 online Forums

Detect

Segment

Classify

Pose

OBB

Track

▼ Detection (COCO)

Explore the [Detection Docs](#) for usage examples. These models are trained on the [COCO dataset](#), featuring 80 object classes.

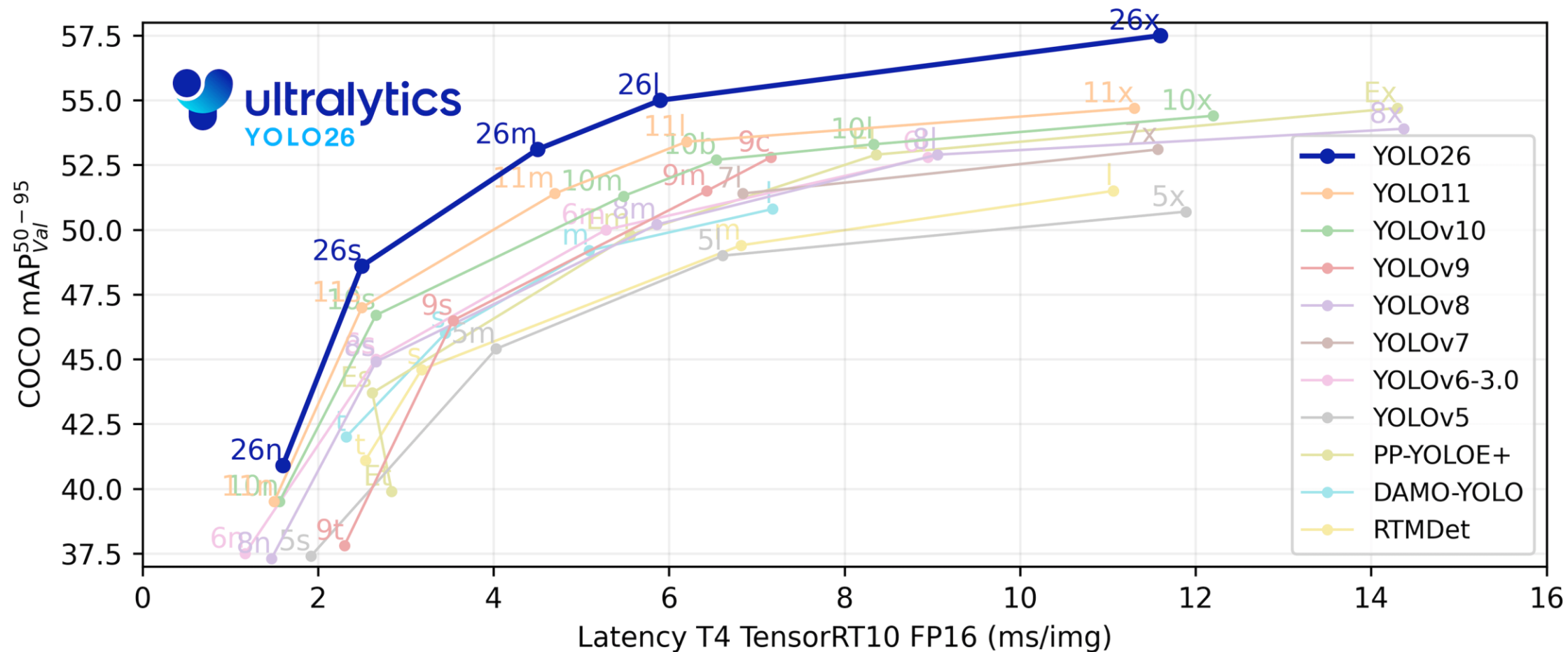
| Model                   | size (pixels) | mAP <sup>val</sup> 50-95 | mAP <sup>val</sup> 50-95(e2e) | Speed CPU ONNX (ms) | Speed T4 TensorRT10 (ms) | params (M) | FLOPs (B) |
|-------------------------|---------------|--------------------------|-------------------------------|---------------------|--------------------------|------------|-----------|
| <a href="#">YOLO26n</a> | 640           | 40.9                     | 40.1                          | 38.9 ± 0.7          | 1.7 ± 0.0                | 2.4        | 5.4       |
| <a href="#">YOLO26s</a> | 640           | 48.6                     | 47.8                          | 87.2 ± 0.9          | 2.5 ± 0.0                | 9.5        | 20.7      |
| <a href="#">YOLO26m</a> | 640           | 53.1                     | 52.5                          | 220.0 ± 1.4         | 4.7 ± 0.1                | 20.4       | 68.2      |
| <a href="#">YOLO26l</a> | 640           | 55.0                     | 54.4                          | 286.2 ± 2.0         | 6.2 ± 0.2                | 24.8       | 86.4      |
| <a href="#">YOLO26x</a> | 640           | 57.5                     | 56.9                          | 525.8 ± 4.0         | 11.8 ± 0.2               | 55.7       | 193.9     |

## Ultralytics YOLO 文档

<https://docs.ultralytics.com/zh/>

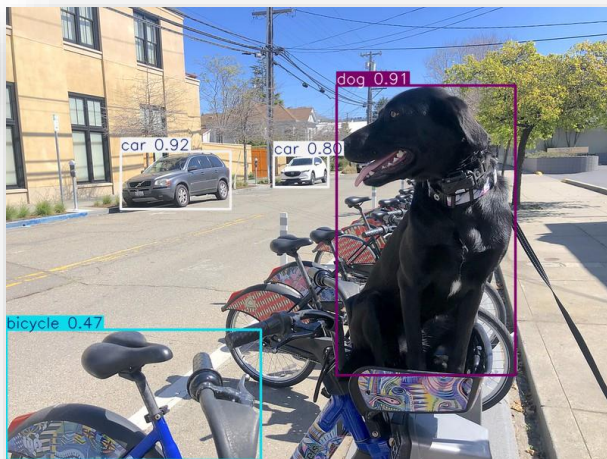
<https://github.com/ultralytics/ultralytics>

# Ultralytics YOLO 執行效能比較



資料來源：<https://github.com/ultralytics/ultralytics>

# Intel OpenVINO YOLO Colab範例



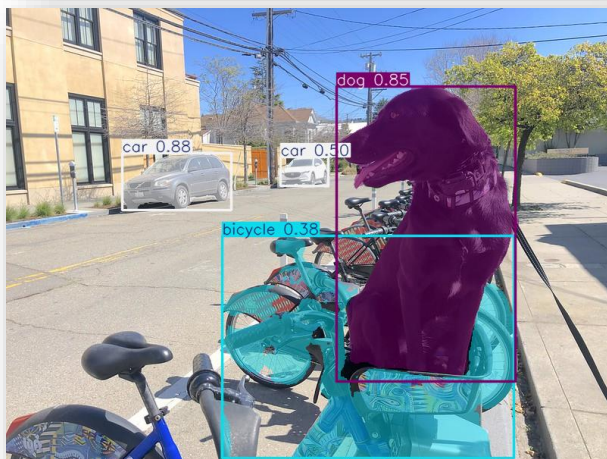
## 物件偵測

YOLOv8  
YOLOv11  
YOLOv12  
**YOLO26**



## 姿態估測

YOLOv8  
YOLOv11  
**YOLO26**



## 實例分割

YOLOv8  
YOLOv11  
YOLOE-26  
**YOLO26**



## 旋轉物件偵測

YOLOv8  
**YOLO26**

[https://openvinotoolkit.github.io/openvino\\_notebooks/?search=YOLO](https://openvinotoolkit.github.io/openvino_notebooks/?search=YOLO)

# 範例6-2:yolov26-object-detection-pytorch.ipynb

## 1. 取得並安裝必要函式庫

```
import platform

if platform.system() == "Darwin":
    %pip install -q "numpy<2.0.0"

#%pip install -qU "openvino>=2026.0.0" "nncf>=3.0.0"
%pip install -q "torch==2.10" torchvision --extra-index-url https://download.pytorch.org/whl/cpu
%pip install -q "ultralytics==8.4.22" tqdm opencv-python

... 1.2/1.2 MB 20.6 MB/s eta
```

## 2. 選擇 Yolo26 模型

可選擇不同大小的模型，包括 "yolo26n", "yolo26s", "yolo26m", "yolo26l", "yolo26x"，預設為 "yolo26n"

```
import ipywidgets as widgets

model_id = ["yolo26n", "yolo26s", "yolo26m", "yolo26l", "yolo26x"]

model_name = widgets Dropdown(options=model_id, value=model_id[0], description="Model")

model_name

... Model yolo26n
```

## 3. 指定物件偵測推論模型及裝置並進行推論

下載 YOLO 預訓練模型，並指定使用 CPU 推論

指定並下載影像並執行推論

顯示推論結果

```
from PIL import Image
from ultralytics import YOLO

DET_MODEL_NAME = model_name.value

det_model = YOLO(f"{DET_MODEL_NAME}.pt")
det_model.to("cpu")
label_map = det_model.model.names

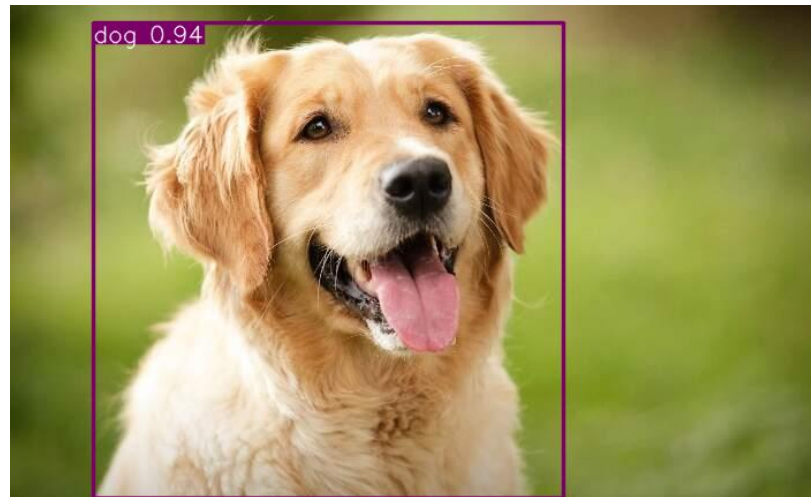
#res = det_model([IMAGE_PATH])
res = det_model("https://www.nylabone.com/~media/project/oneweb/nylabone/images/dog101/10-intelligent-dog-breeds/golden-retriever-tongue-out.jpg")
Image.fromarray(res[0].plot()[0, :, :]).show()
```

Creating new Ultralytics Settings v0.0.6 file ☒ View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json' Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs\_dir=path/to/dir'. For help see <https://docs.ultralytics.com/quickstart/#ultralytics-settings>.

Downloading

<https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo26n.pt> to 'yolo26n.pt': 100% 5.3MB 62.8MB/s 0.1s Downloading

<https://www.nylabone.com/~media/project/oneweb/nylabone/images/dog101/10-intelligent-dog-breeds/golden-retriever-tongue-out.jpg> to 'golden-retriever-tongue-out.jpg': 100% 24.3KB 63.7MB/s 0.0s image 1/1 /content/golden-retriever-tongue-out.jpg: 416x640 1 dog, 304.2ms Speed: 13.3ms preprocess, 304.2ms inference, 5.7ms postprocess per image at shape (1, 3, 416, 640)



[https://colab.research.google.com/github/OmniXRI/NTUST\\_EdgeAI\\_2026/blob/main/Ch6\\_Image\\_Reognition/yolov26-object-detection-pytorch.ipynb](https://colab.research.google.com/github/OmniXRI/NTUST_EdgeAI_2026/blob/main/Ch6_Image_Reognition/yolov26-object-detection-pytorch.ipynb)

# 範例6-3:yolov26-object-detection-openvino-ir.ipynb

## 使用 OpenVINO™ 轉換 YOLO26 模型成 IR 格式並進行物件偵測推論

```
# object detection model
det_model_path = Path(f"{DET_MODEL_NAME}_openvino_model/{DET_MODEL_NAME}.xml")

if not det_model_path.exists():
    det_model.export(format="openvino", dynamic=False, half=True, end2end=True)

... Ultralytics 8.4.22 Python-3.12.13 torch-2.10.0+cpu CPU (Intel Xeon CPU @ 2.20GHz)
    ProTip: Export to OpenVINO format for best performance on Intel hardware. Learn more at https://docs.ultralytics.com/integrations/openvino/

PyTorch: starting from 'yolo26n.pt' with input shape (1, 3, 640, 640) BCHW and output shape(s) (1, 300, 6) (5.3 MB)

OpenVINO: starting export with openvino 2026.1.0-21367-63e31528c62-releases/2026/1...
OpenVINO: export success 5.1s, saved as 'yolo26n_openvino_model/' (5.1 MB)

Export complete (5.6s)
Results saved to /content
Predict:      yolo predict task=detect model=yolo26n_openvino_model imgsz=640 half
Validate:     yolo val task=detect model=yolo26n_openvino_model imgsz=640 data=/home/lq/codes/ultralytics/ultralytics/cfg/datasets/coco.yaml half
Visualize:    https://netron.app

device = device_widget()

device

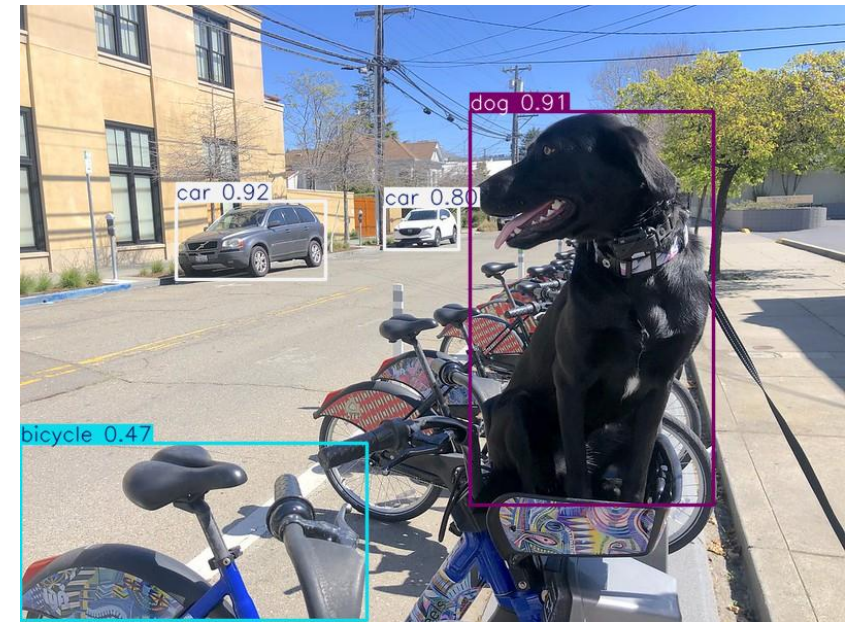
Device: AUTO

det_model = YOLO(det_model_path.parent, task="detect")

res = det_model(IMAGE_PATH, device=f"intel:{device.value.lower()}")
Image.fromarray(res[0].plot()[0, :, :,-1])

Loading yolo26n_openvino_model for OpenVINO inference...
WARNING ⚠ OpenVINO device 'AUTO' not available. Using 'AUTO' instead.
Loading yolo26n_openvino_model for OpenVINO inference...
Using OpenVINO LATENCY mode for batch=1 inference on CPU...

image 1/1 /content/data/coco_bike.jpg: 640x640 1 bicycle, 2 cars, 1 dog, 226.1ms
Speed: 6.1ms preprocess, 226.1ms inference, 0.5ms postprocess per image at shape (1, 3, 640, 640)
```



[https://colab.research.google.com/github/OmniXRI/NTUST\\_EdgeAI\\_2026/blob/main/Ch6\\_Image\\_Reognition/yolov26-object-detection-openvino-ir.ipynb](https://colab.research.google.com/github/OmniXRI/NTUST_EdgeAI_2026/blob/main/Ch6_Image_Reognition/yolov26-object-detection-openvino-ir.ipynb)

# 如何存取虛擬機磁碟檔案到Google Drive

- 在 Colab 中掛載 Google Drive

```
from google.colab import drive  
drive.mount('/content/drive')
```

- 將虛擬機檔案存回 Google Drive 的 /backup\_folder

```
!cp /content/your_file.zip "/content/drive/MyDrive/backup_folder/"
```

- 將 Google Drive /backup\_folder 檔案複製回虛擬機

```
!cp "/content/drive/MyDrive/dataset.zip" /content/  
!unzip /content/dataset.zip
```

- 亦可利用左側檔案夾功能直接複製貼上

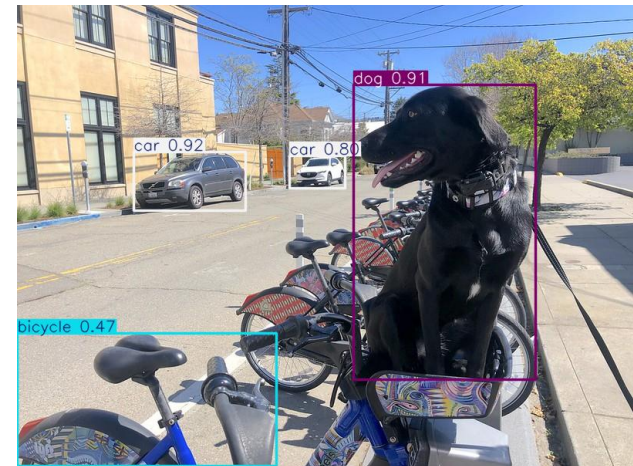
# 範例6-4: yolov26-object-detection-openvino-nncf.ipynb

使用 OpenVINO™ 轉換 YOLO26 模型經 NNCF 優化模型後再進行物件偵測推論

NNCF 提供一套先進的神經網路推論優化演算法，且準確度下降極低。我們將在訓練後（不含微調管線）中使用 8 位元量化來優化模型。

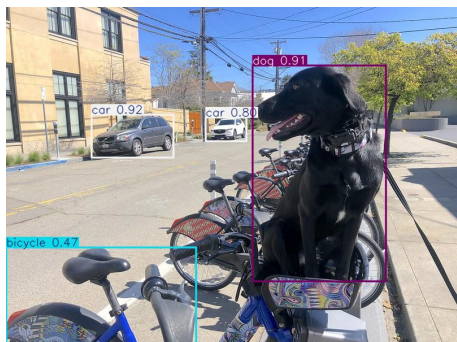
優化過程包含以下步驟：

1. 建立一個用於量化的資料集。
2. 執行 `nncf.quantize` 以獲得最佳化模型。
3. 使用 `openvino.save_model` 函式序列化 OpenVINO IR 模型。



[https://colab.research.google.com/github/OmniXRI/NTUST\\_EdgeAI\\_2026/blob/main/Ch6\\_Image\\_Recognition/yolov26-object-detection-openvino-nncf.ipynb](https://colab.research.google.com/github/OmniXRI/NTUST_EdgeAI_2026/blob/main/Ch6_Image_Recognition/yolov26-object-detection-openvino-nncf.ipynb)

# Yolo26 不同模型處理後推論時間比較



**前處理**：影像色彩、影像尺寸、數值正規化 ...

**推論**：依選擇模型進行物件偵測推論

**後處理**：繪製物件框、標籤、置信度 ...

**Ps.** 雲端推論會視網路及伺服器負擔每次推論時間會有很大不同

| Model Format      | Preprocess | Inference | Postprocess |
|-------------------|------------|-----------|-------------|
| Yolo26<br>PyTorch | 15.2 ms    | 420.2 ms  | 5.9 ms      |
| OpenVINO<br>IR    | 4.4 ms     | 148.0 ms  | 0.4 ms      |
| OpenVINO<br>NNCF  | 5.1 ms     | 111.4 ms  | 0.6 ms      |

26.5%  
(3.77x)

32.5%  
(2.84x)

75.2%  
(1.33x)

[https://colab.research.google.com/github/openvinotoolkit/openvino\\_notebooks/blob/latest/notebooks/yolov26-optimization/yolov26-object-detection.ipynb](https://colab.research.google.com/github/openvinotoolkit/openvino_notebooks/blob/latest/notebooks/yolov26-optimization/yolov26-object-detection.ipynb)

# 範例6-5：OpenVINO YOLO Live Demo

Live demo

[back to top](#) **1**

The following code runs model inference on a video:

```
import collections
import time
from IPython import display
import cv2
import numpy as np

# Main processing function to run object detection.
def run_object_detection(
    source=0,

    # ctrl-c
    except KeyboardInterrupt:
        print("Interrupted")
    # any different error
    except RuntimeError as e:
        print(e)
    finally:
        if player is not None:
            # Stop capturing.
            player.stop()
        if use_popup:
            cv2.destroyAllWindows()
```

```
run_object_detection(
    source=VIDEO_SOURCE,
    flip=True,
    use_popup=False,
    model=det_model,
    device=device.value,
    # video_width=1280,
)
```

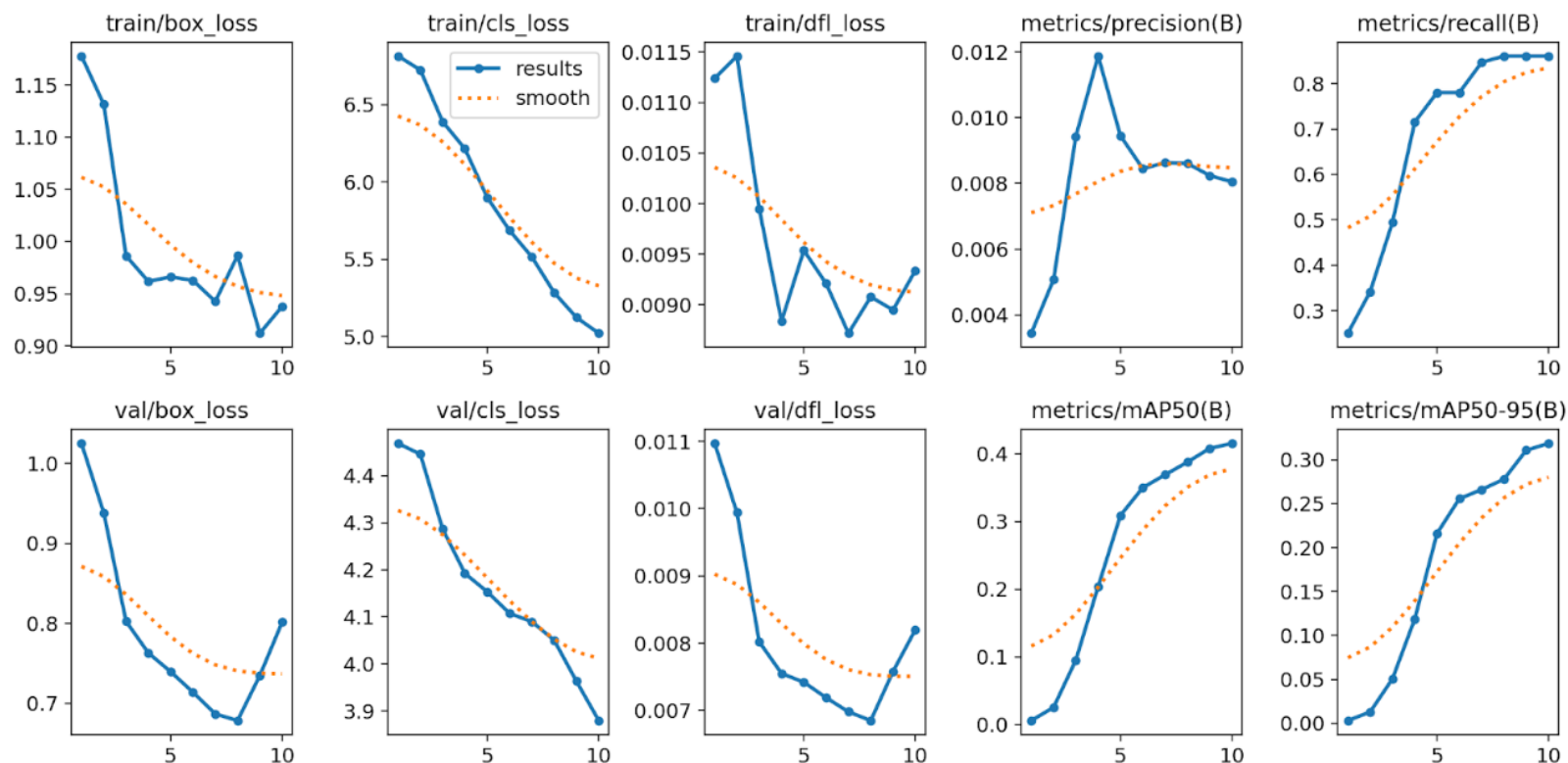
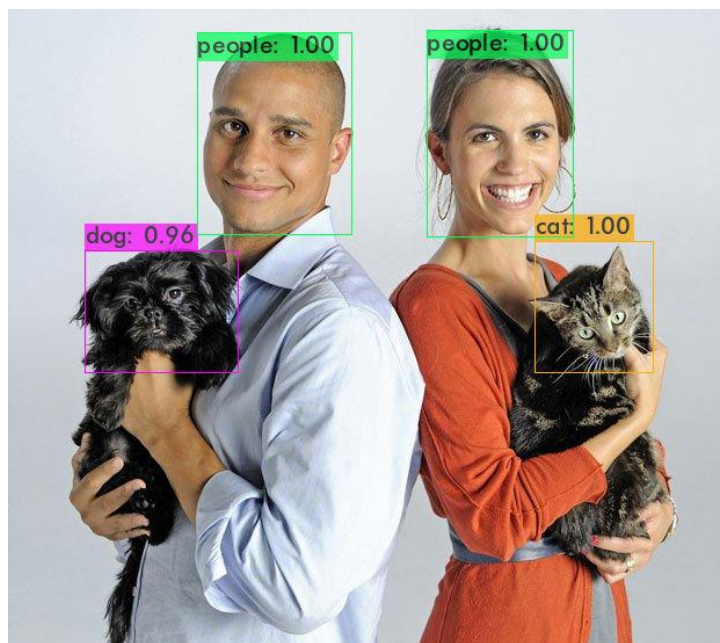
source = 0 表 Webcam,  
但只能在本機執行,  
Colab 上只能執行  
\*.mp4 影片格式



[https://colab.research.google.com/github/openvinotoolkit/openvino\\_notebooks/blob/latest/notebooks/yolov26-optimization/yolov26-object-detection.ipynb](https://colab.research.google.com/github/openvinotoolkit/openvino_notebooks/blob/latest/notebooks/yolov26-optimization/yolov26-object-detection.ipynb)

# 範例6-6：Yolo26 自定義資料集訓練(1/2)

以狗臉、貓臉、人臉偵測為例，共100張已標註影像，80張訓練集，20張驗證集，另備2張測試集。



[https://colab.research.google.com/github/OmniXRI/Yolo26\\_Colab\\_User\\_Datasets/blob/main/yolov26-training-test.ipynb](https://colab.research.google.com/github/OmniXRI/Yolo26_Colab_User_Datasets/blob/main/yolov26-training-test.ipynb)

# 範例6-6：Yolo26 自定義資料集訓練(2/2)

```
from google.colab import drive

drive.mount('/content/drive') # 掛載 Google Drive
```

```
!pip install ultralytics #以命令列方式安裝 Ultralytics Yolo26
```

```
# 下載資料集和相關參數檔
!git clone https://github.com/OmniXRI/Yolo26_Colab_User_Datasets
!mkdir datasets
!unzip Yolo26_Colab_User_Datasets/my_dataset.zip -d datasets
```

```
from ultralytics import YOLO

# 載入 YOLO26 權重
model = YOLO('yolo26n.pt')
```

```
# 開始訓練
results = model.train(
    data='/content/datasets/my_dataset/mydata.yaml', # .資料集設定
    epochs=100, # 訓練次數
    imgsz=640, # 影像尺寸
    batch=-1, # 自動尋找 GPU 極限 batch size
    save_period=50, # 每 50 個 epoch 自動儲存一次備份
    project='/content/drive/MyDrive/YOLO26_Backup', # 直接存入雲端硬碟
    name='my_custom_training', # 存放在 Backup 下的子資料夾名
    exist_ok=True, # 重新啟動時若資料夾存在，不自動更名
    device=0, # 強制使用 GPU
    cache=True # 將圖片緩存至 RAM 加速訓練
)
```

```
from ultralytics import YOLO

# 載入你訓練好的權重 (請根據實際路徑修改)
model = YOLO('/content/drive/MyDrive/YOLO26_Backup/my_custom_training/weights/best.pt')

# 進行推論 (source 可以是圖片路徑、資料夾、甚至是 YouTube 網址)
results = model.predict(source='/content/datasets/my_dataset/test/test01.jpg', conf=0.25, save=True)

# 顯示結果圖 (在 Colab 顯示)
from google.colab.patches import cv2_imshow
import cv2

for r in results:
    img = r.plot() # 繪製置信度大於設定值的物件框
    cv2_imshow(img)
```

[https://colab.research.google.com/github/OmniXRI/Yolo26\\_Colab\\_User\\_Datasets/blob/main/yolov26-training-test.ipynb](https://colab.research.google.com/github/OmniXRI/Yolo26_Colab_User_Datasets/blob/main/yolov26-training-test.ipynb)

# 範例6-7：旋轉物件偵測(OBB)

## YOLO26 Oriented Bounding Boxes Object Detection with OpenVINO™

本範例類似一般 YOLO26 物件偵測，只是模型改為 yolo26~~X~~-obb (~~X~~ 為 n,s,m,l,x)。範例中的模型一樣分為三個階段，一般 PyTorch、OpenVINO IR 格式及經 NNCF優化格式。

可搭配 OpenVINO benchmark\_app 進行更進一步效能分析。



[https://colab.research.google.com/github/openvinotoolkit/openvino\\_notebooks/blob/latest/notebooks/yolov26-optimization/yolov26-obb.ipynb](https://colab.research.google.com/github/openvinotoolkit/openvino_notebooks/blob/latest/notebooks/yolov26-optimization/yolov26-obb.ipynb)

# 補充：人臉辨識（人臉定位、身份辨識）

國立臺灣科技大學  
NATIONAL TAIWAN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Department of Computer Science & Information Engineering  
資訊工程系

## 人工智慧與邊緣運算實務

### 7.3 邊緣智慧案例實作【人臉辨識】

邊緣等級(Edge Level)

Cloud Edge Multi-Access Edge Heavy Edge Medium Edge Mini Edge Micro Edge

資訊工程系 許哲豪 助理教授

國立臺灣科技大學  
NATIONAL TAIWAN UNIVERSITY OF SCIENCE AND TECHNOLOGY

## 7.3 人臉辨識

- 基本介紹
- 評量方式
- OpenCV
- OpenVINO

2023/5/3 CS5149701\_人工智慧與邊緣運算實務\_CH7\_邊緣智慧案例實作\_台科大資工系許哲豪助理教授 2

[https://github.com/OmniXRI/NTUST\\_EdgeAI\\_2023/tree/main/Ch7\\_Implementations/Ch7-3\\_Face\\_Recognition](https://github.com/OmniXRI/NTUST_EdgeAI_2023/tree/main/Ch7_Implementations/Ch7-3_Face_Recognition)

# 參考文獻

---

- 臺灣科技大學資訊工程系 人工智慧與邊緣運算實務 ( CS5149701 )  
<https://omnixri.blogspot.com/p/ntust-edge-ai.html>
- Intel OpenVINO Open Model Zoo Github  
[https://github.com/openvinotoolkit/open\\_model\\_zoo](https://github.com/openvinotoolkit/open_model_zoo)
- Intel OpenVINO Notebooks YOLO 相關範例  
[https://openvinotoolkit.github.io/openvino\\_notebooks/?search=yolo](https://openvinotoolkit.github.io/openvino_notebooks/?search=yolo)
- Ultralytics YOLO 說明文檔  
<https://docs.ultralytics.com/zh/>
- 許哲豪，Ultralytics YOLO26 自定義資料集模型訓練  
[https://github.com/OmniXRI/Yolo26\\_Colab\\_User\\_Datasets](https://github.com/OmniXRI/Yolo26_Colab_User_Datasets)

# 延伸閱讀

---

- Ranjan Sapkota et al., YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection  
<https://arxiv.org/abs/2509.25164>
- Github, open-mmrotate  
<https://github.com/open-mmlab/mmdetection>
- 許哲豪，如何以Google Colab及Yolov4-tiny來訓練自定義資料集—以狗臉、貓臉、人臉偵測為例  
<https://omnixri.blogspot.com/2021/05/google-colabyolov4-tiny.html>